Retrieval-Augmented Generation for Natural Language Processing: A Survey

Shangyu Wu[1,2], Ying Xiong[2†], Yufei Cui[3], Haolun Wu[3], Can Chen[3], Ye Yuan[3], Lianming Huang[1], Xue Liu[2], Tei-Wei Kuo[4], Nan Guan[1], Chun Jason Xue[2]

[1]City University of Hong Kong.
[2]Mohamed bin Zayed University of Artificial Intelligence.
[3]McGill University, Mila.
[4]National Taiwan University.

[†]Corresponding author.

Abstract

Large language models (LLMs) have achieved strong empirical performance in various fields, benefiting from their huge amount of parameters that store knowledge. However, LLMs still suffer from several key issues, such as hallucination problems, knowledge update issues, and lacking domain-specific expertise. The appearance of retrieval-augmented generation (RAG), which leverages an external knowledge base to augment LLMs, mitigates these limitations. This paper presents a systematic review of RAG techniques for natural language processing (NLP), with a focus on retrievers and retrieval fusions. We introduce a novel taxonomy of retrieval fusions, such as query-based, logits-based, latent, and parametric fusion, and provide structured comparisons across accessibility, efficiency, and use cases. The paper further examines RAG applications across diverse NLP tasks, discusses evaluation methodologies and benchmark limitations, and analyzes training paradigms with and without knowledge base updates. Finally, we explore industrial deployment considerations and identify emerging challenges and future directions, including security, efficiency, and graph-based retrieval.

Keywords: Retrieval-augmented generation, natural language processing, vector database, large language model

1 Introduction

Large language models (LLMs) (Touvron et al. 2023a; Mesnard et al. 2024; Jiang et al. 2023a; OpenAI 2023; Zeng et al. 2023) have achieved significant advancements in recent years and have become the cornerstone of various applications in the field of natural language processing (NLP). These LLMs are typically pre-trained on a large amount of natural language corpus and then fine-tuned on the specific downstream tasks' datasets. Recent works (Petroni et al. 2019; AlKhamissi et al. 2022; Meng et al. 2022a; He et al. 2024) demonstrate that the success of LLMs can be explained by the fact that LLMs implicitly store the learned knowledge in the parameters as internal memory and generate responses by retrieving answers from memory. To store more knowledge for better generation performance, existing works generally enlarge the memory capacity by increasing the volume of parameters (Abnar et al. 2022; Brown et al. 2020; Kaplan et al. 2020; Hoffmann et al. 2022).

Although existing LLMs have become a foundational component in modern NLP systems, several challenges still hinder the development of LLMs. One of the most prominent challenges is the hallucination problem (Ji et al. 2023a; Dale et al. 2023; Ji et al. 2023b; Siino and Tinnirello 2024), which refers to the tendency of LLMs to generate responses that are coherent and fluent but factually incorrect. Furthermore, the knowledge update issue poses a significant obstacle. To update the knowledge stored in the LLMs' internal memory (Meng et al. 2022a; Wang et al. 2025a; Zhang et al. 2024), it is necessary to retrain/fine-tune LLMs with new data, which is a costly process. Additionally, general LLMs often lack domain-specific expertise (Zhang et al. 2023a; Singhal et al. 2023a,b; Colombo et al. 2024). Building a domain-specific LLM demands substantial effort in data curation and model adaptation.

To address these challenges, recent works (Lewis et al. 2020; Borgeaud et al. 2022; Guu et al. 2020) have proposed augmenting LLMs with an external knowledge base (KB), known as Retrieval-Augmented Generation (RAG). By supplying the model with relevant, factually grounded information at inference time, RAG enables generation that is informed by verifiable sources rather than relying solely on parametric memory. This approach mitigates the hallucination problem by anchoring outputs in retrieved information. Furthermore, RAG addresses the issue of knowledge staleness by updating the external database allows the model to leverage current information without costly retraining. Similarly, domain-specific expertise can be imparted by constructing and integrating a specialized KB, effectively adapting a general-purpose LLM to specialized tasks. Consequently, RAG plays a pivotal role in enhancing the accuracy, adaptability, and reliability of LLMs across a broad spectrum of applications.

Contributions. This paper presents a systematic and comprehensive survey of RAG techniques for natural language processing. Although several prior surveys have explored this emerging paradigm (Li et al. 2022a; Gao et al. 2023b; Hu and Lu 2024; Zhao et al. 2024; Huang and Huang 2024; Ding et al. 2024), our work offers distinct perspectives and deeper technical insights, as summarized in Table 1. The main contributions of this survey are as follows:

Table 1 Comparison of existing surveys [1] (Li et al. 2022a), [2] (Gao et al. 2023b), [3] (Hu and Lu 2024), [4] (Zhao et al. 2024), [5] (Huang and Huang 2024), and [6] (Ding et al. 2024). ✓ represents that the corresponding survey has introduced the content **in detail**. △ indicates that the corresponding survey **briefly** introduced the content. − represents that the corresponding survey **doesn't mention** the content.

Modules		[1]	[2]	[3]	[4]	[5]	[6]	Ours
Retriever	Building	−	✓	△	△	△	✓	✓
	Querying	△	✓	△	△	△	✓	✓
Retrieval Fusions		△	△	✓	✓	△	✓	✓
Generator		△	△	✓	✓	△	✓	✓
RAG on NLP tasks		✓	△	✓	△	−	△	✓
RAG Benchmarking		−	✓	✓	✓	✓	−	✓
RAG Training/Update		−	△	△	△	△	✓	✓
RAG Tools		−	−	−	✓	−	✓	✓
Tutorial Codes		−	−	−	−	−	−	✓

1. We provide an end-to-end, system-oriented decomposition of RAG for NLP, complemented with algorithm-level walkthrough and time/space complexity analyses to bridge concepts and implementable pipelines[1].
2. We propose a comprehensive taxonomy of retrieval fusion (query-, logits-, latent-, and parametric fusion) and a structured comparison across accessibility, efficiency, implementation complexity, and use cases, including practical insights on hybrid fusion designs.
3. We synthesize RAG training and knowledge-update strategies under settings with and without datastore updates, linking retriever/generator optimization with parameter-efficient knowledge injection workflows.
4. We comprehensively introduce the applications of RAG across a wide range of NLP tasks and RAG evaluation organizing metrics and methodologies.
5. We also extend the survey toward practical scenarios with tooling/ecosystem and deployment considerations alongside emerging directions such as security/privacy or GraphRAG.

The structure of this paper is organized as follows and outlined in Figure 1. Section 2 provides an overview of RAG. Sections 3 and 4 then delve into the technical details of retrievers and retrieval fusion mechanisms, respectively. Section 5 examines the generator component. Subsequently, Section 6 reviews the techniques employed in representative NLP tasks. Section 7 introduces evaluation methodologies and benchmarks for RAG systems. Section 8 discusses training paradigms for RAG, distinguishing between configurations with and without dynamic knowledge integration. Section 9 surveys RAG deployment in practical NLP scenarios. Section 10 identifies promising directions for future research, and Section 11 concludes the paper.

[1] Tutorial codes are available on https://github.com/luffy06/RAG-Tutorials

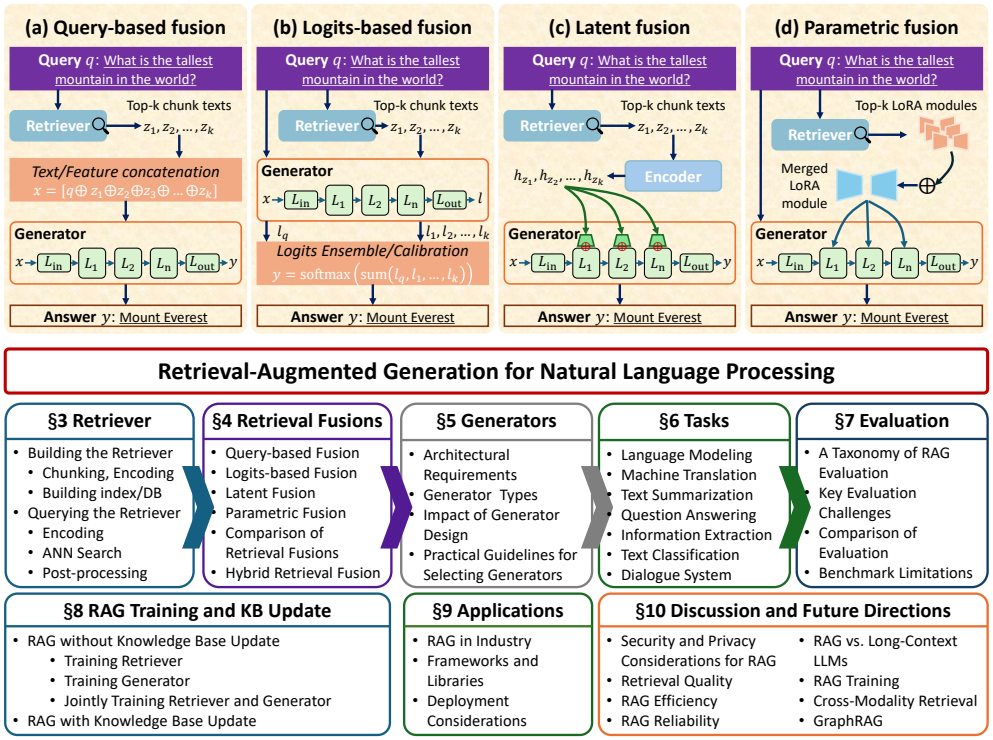

Fig. 1 The overview of retrieval-augmented generation for natural language processing. The inputs as queries are fed into both the retriever for retrieval knowledge and the generator for outputs. There are four kinds of retrieval fusions, including query-based fusion, logits-based fusion, latent fusion, and parametric fusion.

2 Overview

This section gives an overview of RAG for NLP. An RAG system utilizes the external knowledge base $\boldsymbol{D}$ to enhance the generation system. Taking external documents as an example, $\boldsymbol{D}$ consists of external documents, each of which contains a set of chunks $c_i \in \boldsymbol{C_i}$. These chunks are transformed into vector embedding using an embedding model. When inputting a query q, which is embedded as a vector, the retriever in the RAG system retrieves top-k chunks $Z_q = \{z_1, z_2, \ldots, z_k\}$ most relevant to the query q. The RAG system can use different retrieval fusion to fuse the retrieved chunks, which is discussed in Section 4. The overall process is formulated as follows:

$$\texttt{Retriever}(q, D) \rightarrow Z_q \tag{1}$$

$$\texttt{Generator}(\texttt{Retrieval Fusion}(q, Z_q)) \rightarrow answer \tag{2}$$

As shown in the above equations, the RAG system typically consists of three modules: the retriever, the generator, and the retrieval fusions.

Retriever module usually comprises three components: an encoder for encoding inputs into embeddings, an efficient indexing that supports approximate nearest

neighbor (ANN) search, and a vector database for storing external knowledge in the form of key-value pairs. The main challenge in the retriever module is *finding the optimal trade-off between retrieval efficiency and retrieval quality*. The retrieval efficiency refers to how fast the relevant information can be obtained, which involves accelerating encoding, efficient indexing, batch querying in the vector database, etc. The retrieval quality refers to how relevant the information can be retrieved, which involves chunk representation learning, advanced ANN search algorithms, etc.

Retrieval Fusions aim to leverage the retrieved information to augment the generation. These retrieval fusions can be categorized into four major types: query-based fusion, logits-based fusion, latent fusion, and parametric fusion. The query-based fusion augments inputs with retrievals before feeding them into the generators. The logits-based fusion focuses on the output logits of generators and fuses the retrieval logits for more robust logits. The latent fusion refers to introducing retrieval representations into the latent representations of generators, thus implicitly improving the models' performance. The parametric fusion refers to the process of encoding external knowledge into Low-Rank Adaptation (LoRA) modules, which are subsequently merged into the model's weights to inject the retrieved information. Specifically, the top-k most relevant LoRA modules are selected and integrated, thereby augmenting the model with task-specific knowledge without modifying its original parameters.

Generator modules in RAG systems fall into three categories. Proprietary LLMs (e.g., GPT (Radford et al. 2018, 2019; Brown et al. 2020; OpenAI 2023), Gemini (Anil et al. 2023; Mesnard et al. 2024; Reid et al. 2024)) offer strong capabilities but cannot be deployed locally. Open-weight pretrained LLMs, such as Mistral (Jiang et al. 2023a), Qwen (Yang et al. 2024a,b), DeepSeek (DeepSeek-AI 2025), and Llama (Touvron et al. 2023a,b; Team 2024), provide flexibility for local deployment. Retrieval-aware generators (e.g., RETRO (Borgeaud et al. 2022; Wang et al. 2024a), Enc-Dec (Li et al. 2022b)) are specifically designed to incorporate retrieved information through dedicated fusion mechanisms.

The workflow of the RAG system involves three steps: 1) retrieving the relevant information from the external KB based on the given input query; 2) fusing the retrieved information with inputs or intermediate states based on the retrieval fusion; 3) making predictions by generators based on the input query and corresponding retrievals.

3 Retriever

Figure 2 shows the two stages for using the retriever, which involve first building the retriever and then querying the retriever. The following sections will introduce details about each stage.

3.1 Building the Retriever

This section will explain how to build a retriever using a large natural language corpus. As shown in Figure 2 (a), the process involves three steps: chunking corpus, encoding chunks, and building the vector database. Specifically, building the vector database includes building the ANN index and storing the data with key-value pairs.

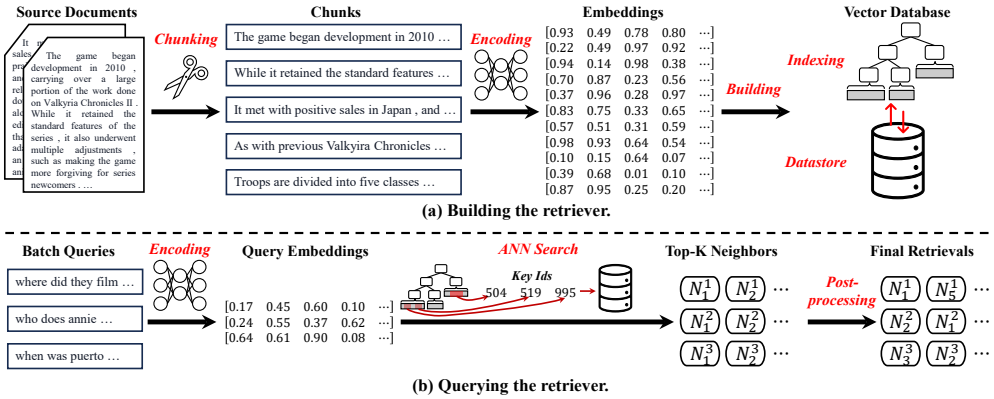

Fig. 2 Two stages of using the retriever.

3.1.1 Chunking Corpus

Chunking techniques generally refer to dividing large documents into small text chunks (Muszynska 2016; Ishiwatari et al. 2017; Gong et al. 2020; Borgeaud et al. 2022; Chen et al. 2022a), which is an indispensable key step in the process of building the retriever. The intuitions behind chunking techniques are, (1) The texts or embeddings used for the indexing should be semantically independent, containing one core idea for models to encode. Short texts are more likely to be ambiguous, for example, the word "apple" can refer to a fruit or a company. (2) Encoding a long sequence document would result in considerable resource overhead when using existing transformer-based models, while processing shorter text chunks can significantly accelerate the encoding process and save memory costs. Therefore, the main challenge of the chunking techniques is to find the best chunking size to make a better trade-off between text semantics and encoding efficiency.

To solve the above challenge, three key points need to be considered when determining the chunking size:

1. **Task's preference.** Different tasks benefit from different chunk sizes based on their information granularity. For example, for question answering tasks, they prefer short sentences, where the chunk size of 128 tokens is enough, as representative QA tasks only have 100-200 tokens at most per question (Kwiatkowski et al. 2019; Mihaylov et al. 2018). However, for summarization tasks, they are usually long documents, where the choice for chunk size can be 512 tokens due to lengthy documents (Narayan et al. 2018).
2. **Encoder's preference.** Different encoder models have varying encoding capabilities on texts with different lengths. For example, models in the sentence-transformer (Reimers and Gurevych 2019) behave better on a single sentence, while the text-embedding-ada-002 (OpenAI 2022) is good at longer texts. The choice of chunk size usually depends on the max input sequence length of the encoder model, e.g., 512 tokens for BERT-based models.

3. **Query's preference.** The length of the user's queries should be aligned with the chunking size, which implicitly aligns the amount of contextual information in chunks with that in queries, thus improving the relevance between queries and retrievals. For example, a retrieval database built on short phrases may be useless for queries with long documents.

Overall, there is no golden rule for determining the chunking size, and it depends on the specific RAG scenarios.

There are basically three types of chunking techniques, including **the chunking with fixed length**, **the semantic chunking**, and **the content-based chunking**. Chunking with a fixed length is the simplest way to split documents sequentially using a length hyperparameter. The semantic chunking cuts documents based on semantics, such as the period character or the newline character that represents the end of the sentence. Existing state-of-the-art natural language processing toolkits, such as NLTK (NLTK 2001) and spaCy (explosion 2016), have provided convenient sentence-cutting methods. The content-based chunking segments documents according to the unique structural characteristics. For example, electronic medical records can be easily segmented based on the sections, or programming codes can be segmented based on function blocks.

Before chunking and encoding, text preprocessing (e.g., normalization, removing boilerplate/HTML artifacts, deduplication, and consistent handling of casing/punctuation) can influence the quality of chunk representations. Prior studies (Siino et al. 2024) show that preprocessing choices can measurably affect downstream performance for both Transformer-based models and traditional models, suggesting that corpus preparation should be treated as an important knob when optimizing retrieval quality and robustness in RAG.

3.1.2 Encoding Chunks

Encoding refers to numericalizing textual chunks as vector representations (embeddings). These embeddings generally capture the semantics of the chunks, enabling the retriever to perform similarity searches based on content relevance rather than just keyword matching.

According to the sparsity of the embeddings, there are two kinds of encoding methods, i.e., **sparse encoding** and **dense encoding**. The sparse encoding represents text by creating high-dimensional vectors where most elements are zero. The basic sparse encoding is one-hot encoding (Harris and Harris 2010), which represents a word with a high-dimensional vector as large as the vocabulary table size but only marks the value corresponding to the presence of the word as one. The embeddings produced by such encodings are called the one-hot vector. Other common sparse encodings include:

1. **Bag of Words (BoW)** (Harris 1954). This encoding improves one-hot encoding by replacing the zero-one counting with the frequency counting. However, BoW ignores the syntax and word order in the documents and focuses on statistical information, thus only expressing limited semantics.
2. **Term Frequency-Inverse Document Frequency (TF-IDF)** (Rajaraman and Ullman 2011). This encoding not only counts the occurrence (frequency) of

each word but also adjusts these counts based on how common the word is across all documents (inverse document frequency). TF-IDF helps emphasize words that are more descriptive of the document's content.

3. **BM25** (Robertson and Zaragoza 2009) is a probabilistic ranking algorithm used in information retrieval to estimate the relevance of documents to a search query by balancing term frequency, inverse document frequency, and document length normalization, ensuring robust scoring even for long or short documents. BM25 focuses on lexical matches and is computationally efficient, making it a cornerstone of traditional search engines.

Sparse encoding is an efficient way to encode textual chunks. However, such encoding methods may not capture deeper semantic meanings well.

The dense encoding generates vectors where each dimension can capture a range of semantic features, and most elements are non-zero floating points. The dense embeddings are generally produced by (deep) neural network models,

1. **BERT (Devlin et al. 2019) and Variants.** Bidirectional Encoder Representation from Transformers (BERT) is a typical pre-trained transformer model, generating dense semantic embeddings that capture the contextual information. Other BERT variants, such as RoBERTa (Liu et al. 2019), DistilBERT (Sanh et al. 2019), and ELECTRA (Clark et al. 2020), further improve the semantic representations with advanced learning techniques.
2. **Siamese Encoders.** This is a type of neural network designed to learn the similarity between inputs, which is usually trained with contrastive learning. Existing state-of-the-art siamese encoders are DPR (Karpukhin et al. 2020), SimCSE (Gao et al. 2021), Contriever (Izacard et al. 2022).
3. **LLM-based Encoders.** This type of encoder benefits from the powerful representation capability of LLMs. LLMs, which contain billions of parameters and are pre-trained on vast amounts of data covering a wide range of topics, have advanced semantic language understanding capabilities. Typical LLM-based encoders are text-embedding-ada-002 (OpenAI 2022), bge-embedding (Xiao et al. 2023), mxbai-embedding (Lee et al. 2024), MedCPT (Jin et al. 2023).

Compared to sparse encoding, dense encoding leverages deep neural networks, especially transformers (Vaswani et al. 2017), to capture broader linguistic and semantic information. Dense encoding is widely used in most representation scenarios. Moreover, some works also utilized hybrid methods to encode text for leveraging both lexical and semantic information (Lewis et al. 2022; Luo et al. 2023).

3.1.3 Building the Index

Indexing in the vector database aims to accelerate the search process for data similar to high-dimensional query embedding. Unlike common indexing in databases, indexing in the vector database mainly focuses on supporting efficient ANN search (Johnson et al. 2021; Douze et al. 2024; Guo et al. 2020; Li et al. 2023b; Fu et al. 2024) rather than transaction operations like insertion, deletion, and update. The key challenge of indexing is making a good trade-off between search quality and search efficiency. To solve the challenge, there are various specific optimizations in both algorithmic

aspects and systematic aspects to be explored, including choices of similarity metrics, dimension reduction on embeddings, advanced ANN indexing, system-level optimizations, hardware-aware optimization, and so on. Here we focus on the optimizations that most strongly affect search quality and efficiency.

Choice of Similarity Metrics. The similarity metrics are the basic components in the retriever, which measure the degree of relevance between query embeddings and chunk embeddings. The similarity metrics would affect the search quality. Typical similarity metrics include cosine similarity, Euclidean similarity, and Manhattan distance.

Dimension Reduction on Embeddings. Reducing the dimensionality of embeddings can improve search efficiency but at the risk of harming the semantic representations. The basic but effective dimension reduction is the principal component analysis (PCA). The PCA is a simple statistical technique that transforms the original data into a new coordinate system while retaining the most important features. Another popular and advanced dimension reduction is locality-sensitive hashing (LSH). LSH significantly reduces the dimensionality by mapping the data into buckets but preserves the similarity of the original input data. The intuition behind LSH is that the nearest neighbors will be mapped into the same buckets. Unlike LSH, product quantization (PQ) (Jégou et al. 2011) is another popular and effective DR technique for ANN search. The core idea of the PQ is to divide the high-dimensional space into smaller, independently quantized subspaces. Each subspace creates a codebook of different quantized integers to form the representative and compact vectors. The above techniques enable efficient storage and fast approximate search but may lose semantic information. Recent work (Chevalier et al. 2023) proposed a new technique named AutoCompressor that reduces the dimension of embeddings by compressing the original context into semantically shorter embeddings.

Advanced ANN Indexing. ANN Indexing generally refers to the methods or structures used to organize and manage data so that the approximate-nearest-neighbor search process is optimized for retrieval quality and retrieval efficiency. This paper will introduce several advanced ANN indexing techniques.

1. **The InVerted File system with Product Quantization (IVFPQ)** (Douze et al. 2024) is a simple but effective indexing framework that combines two powerful techniques to enable an efficient and scalable ANN search process. The main idea of IVFPQ is first to cluster the data for coarse-grained partition and then to compress the data within each cluster into sub-vectors for fine-grained quantization. The coarse-grained clustering (the IVF component) significantly reduces the search space, while the fine-grained quantization (the PQ component) ensures a high retrieval performance.
2. **The Hierarchical Navigable Small World (HNSW)** (Malkov and Yashunin 2020) uses a hierarchical graph structure to perform ANN search in high-dimensional spaces efficiently. Specifically, HNSW treats high-dimensional vectors as nodes and connects them with their nearest neighbors. The multi-layer graph structure is determined probabilistically to ensure fewer nodes at higher layers for efficient search.

Algorithm 1 Building the retriever.

Input: A natural language corpus $D = \{d_1, \ldots, d_n\}$ for building the KB, an encoder $\mathcal{E}$ for encoding chunks.

Output: The index $\mathcal{I}$ and the key-value store $\mathcal{S}$.

1: $\mathcal{K} = \{\}, \mathcal{V} = \{\}$;
2: **for** $d_i \in D$ **do**
3: $c_i^1, \ldots, c_i^m = Chunk(d_i)$; /* Split each data d_i */
4: **for** j from 1 to m **do**
5: $e_i^j = \mathcal{E}(c_i^j)$; /* Encode each chunk c_i^j */
6: Add e_i^j into $\mathcal{K}$ and $c_i^j + c_i^{j+1}$ into $\mathcal{V}$; /* Take next chunk as an example */
7: The $\mathcal{K}$ and $\mathcal{V}$ persist in the storage (e.g., SSD) if necessary;
8: **end for**
9: **end for**
10: Build the index $\mathcal{I}$ with $\mathcal{K}$;
11: Store $\mathcal{K}$ and $\mathcal{V}$ into the key-value store $\mathcal{S}$;
12: **return** $\mathcal{I}$ and $\mathcal{S}$;

3. **Tree-based Indexing** aims to organize high-dimensional vectors in tree-liked structures, such as KD-Trees (Ram and Sinha 2019), Ball Trees (Huang and Tung 2023) and VP-Trees (Liu and Wei 2015). Typical tree-based indexing is Approximate Nearest Neighbors Oh Yeah (Annoy) (Spotify 2017), which uses a forest of trees built based on random projections to separate the vector space into multiple hyperplanes for efficient ANN search.

3.1.4 Building the KB with Key-Value Pairs

The vector database used in the KB is a specialized database that stores and manages data as a collection of key-value pairs, where keys are the unique identifiers of high-dimensional embeddings and values are the domain-specific knowledge. Since the amount of data stored in the KB may be quite large, the storage engine, such as LMDB (LMDB 2014) or RocksDB (Facebook 2013), should be capable of efficient retrieval and data persistence. A key design decision in the knowledge base is what should be stored as values. For example, for question-answer tasks, when adding retrievals to prompts, the naive but effective way is to store the question embedding as the key and question-answer pairs as the value. This can help the generation process as retrievals are used as demonstrations for models. Recent works have proposed various state-of-the-art vector databases, including the indexing and the knowledge base, such as Milvus (Wang et al. 2021a; Guo et al. 2022), FAISS (Douze et al. 2024; Johnson et al. 2021), LlamaIndex (Liu 2022), etc.

3.1.5 Code Demonstrations

Algorithm 1 shows detailed steps to build the retriever. Lines 2-8 present the chunking and the encoding process for a natural language corpus containing multiple documents. In line 6, algorithm 1 takes the concatenation of the current chunk and the next chunk as the value. Notably, the choice of value can vary for different tasks. Another practical issue is that the memory cost of all keys and values may exceed the memory

Algorithm 2 Query the retriever.

Input: A query input q, an encoder $\mathcal{E}$ for encoding chunks, the index $\mathcal{I}$, the key-value store $\mathcal{S}$, the parameter k.

Output: Top-k nearest neighbor knowledge.

$e = \mathcal{E}(q)$;
$\{idx_1, \ldots, idx_k\} = \mathcal{I}.Search(e, k)$; /* Search the top-$k$ nearest neighbors */
$\{v_1, \ldots, v_k\} = \mathcal{S}.Fetch(\{idx_1, \ldots, idx_k\})$; /* Fetch the values of the neighbors */
$\{v_{j_1}, \ldots, v_{j_k}\} = PostProcess(\{v_1, \ldots, v_k\})$
return $\{v_{j_1}, \ldots, v_{j_k}\}$;

capacity of the server in a practical scenario. Thus, it is recommended that the keys and values persist in the storage if necessary. The time complexity is $O(n \cdot (C_{\text{chunk}} + m \cdot C_{\text{encode}}) + C_{\text{build\_index}})$, where C_{chunk}, C_{encode}, and $C_{\text{build\_index}}$ are the cost of chunking one document, encoding one chunk, and building the index for all chunks. The space complexity is $O(M \cdot d)$, where M is the number of all chunks and d is the feature dimension of the encoder.

3.2 Querying the Retriever

This section will explain how to query the pre-built retriever, which basically includes three steps as shown in Figure 2(b): encoding queries, ANN search, and post-processing.

3.2.1 Encoding Queries and ANN Search

To align with the pre-built embedding space, the retriever uses the same encoder to encode queries during the querying stage. The ANN search leverages the pre-built indexing and vector database to find similar data via ANN searching algorithms and then retrieves the corresponding values.

Searching the index refers to searching the pre-built index, finding the top-k nearest neighbors, and returning the unique identifiers of the k nearest neighbors. The nearest neighbor search process depends on indexing algorithms or structures. Taking IVFPQ as an example, the search process first compares the query embedding with cluster embeddings and selects several candidate clusters for further search. Then, within each cluster, the search process performs the same product quantization on the query embedding and finds the top-k nearest neighbors based on the distance. Finally, the search process merges all nearest neighbor candidates and re-orders all candidates for the final top-k nearest neighbors.

Retrieving values from vector database fetches the corresponding values based on the nearest key identifiers.

3.2.2 Post-Processing

The post-processing involves a set of techniques after the initial retrieval step. These techniques aim to refine, enhance, or adapt the retrievals based on the specific task objectives. This section will list some typical post-processing techniques.

Algorithm 3 Query-based Fusions.

Input: A query input q, top-k nearest neighbor knowledge $\{v_1, \ldots, v_k\}$, an encoder $\mathcal{E}_f$ and a decoder $\mathcal{D}_f$ for feature concatenation, the generator $\mathcal{G}$ for text concatenation.
Output: Generated response y.

1: **if** Use the text concatenation **then**
2: $x = v_1 \oplus \ldots \oplus v_k \oplus q$; /* Concatenate neighbor texts and query */
3: $y = \mathcal{G}(x)$;
4: **else**
5: $e_q = \mathcal{E}_f(q), e_{v_j} = \mathcal{E}_f(v_j), j \in \{1, \ldots, k\}$;
6: $e_x = e_q \oplus e_{v_1} \oplus \ldots \oplus e_{v_k}$; /* Concatenate embeddings of neighbors and query */
7: $y = \mathcal{D}_f(e_x)$
8: **end if**
9: **return** y;

Reranking aims to reorder the retrieved information based on task-specific objectives. The intuition is that the knowledge is retrieved based on task-agnostic metrics, such as Euclidean distance. Existing reranking methods (Chuang et al. 2023; Hossain et al. 2020; Lazaridou et al. 2022; Vu et al. 2024) mostly design different architectures or strategies to reorder the retrieved information.

3.2.3 Code Demonstrations

After building the retriever, this section demonstrates the detailed steps of querying the retriever to obtain the top-k nearest neighbor knowledge in algorithm 2, including encoding the query (line 1), performing the approximate nearest neighbor search (line 2), and fetching the knowledge for fusion (line 3). These three steps depend on the specific APIs of encoders, indexing, and the vector database. After obtaining the top-k retrievals, optimizations for post-processing are applied (line 4). The time complexity is $O(C_{\text{encode}} + C_{\text{ann}} + k \cdot C_{\text{IO}} + C_{\text{post}})$, where C_{ann}, C_{IO}, and C_{post} are the cost of performing ANN search, fetching neighbor values from the disk, and post-processing. The space complexity is $O(k \cdot d)$.

4 Retrieval Fusions

Retrieval fusions refer to how to leverage the retrieved information to improve generators' performance. Basically, there are four types of retrieval fusions: query-based fusions, logits-based fusions, latent fusions, and parametric fusions.

4.1 Query-based Fusion

The simplest and most direct retrieval fusion is query-based fusion, which integrates the retrieved information with input queries to generate responses. The query-based fusion can be further categorized into two sub-classes according to the type of concatenated information, i.e., text concatenation and feature concatenation.

Text concatenation involves performing query-based fusion with raw texts, making it particularly suitable for proprietary LLMs like GPT-4. These models function as black-box systems with limited interaction capabilities, typically offering only API

access to users. Existing works (Guu et al. 2020; Lewis et al. 2020; Ram et al. 2023b) directly concatenate the input with the top-k retrieved sentences/documents to form the query for generators. To better use the in-context learning capability of LLMs, some works (Fabbri et al. 2020; Wang et al. 2022; Li et al. 2023d; Vu et al. 2024) design effective prompt templates to integrate retrieved information and inputs. To address the issue of lengthy inputs after concatenating retrievals, recent studies (Lyu et al. 2023; Xu et al. 2024; Arefeen et al. 2024; Wang et al. 2023d; Liu et al. 2023a; Chen et al. 2026) have introduced methods for assigning importance weights to elements within the retrieved information base and filtering out less relevant contexts based on these weights.

The feature concatenation involves merging the encoded retrievals with the input features. A simple yet effective approach is FID (Izacard and Grave 2021), which first encodes the retrieved passages into sparse or dense representations and then takes the concatenated features as the input for a generator. The state-of-the-art performance of the FID demonstrates the efficacy of feature concatenation. The follow-up works (Sachan et al. 2021; Guo et al. 2023; de Jong et al. 2023b; Izacard et al. 2023; Liu et al. 2023b) further improve the FID by jointly tuning the retriever and the encoder, thereby enhancing the representations of retrieved information. Besides, Chen et al. (Chen et al. 2022b) concatenate the representations of related knowledge as demonstrations for prompt learning, yielding better generalization.

Algorithm 3 presents how to leverage query-based fusions to fuse retrieved information. For those using text concatenation (Guu et al. 2020; Ram et al. 2023b), algorithm 3 first concatenates the retrieved texts and inputs (line 2), then feeds the concatenated input into the generator. Notably, since there is a limit to the maximum input length of existing language models, concatenating too many retrievals would result in a truncation of the concatenated input, which may cut the given input. Therefore, designing the prompt template is the key step for this branch of work. For those using feature concatenation (Izacard and Grave 2021; Guo et al. 2023), algorithm 3 first leverages an encoder to obtain the feature (line 5), then concatenates the feature of input and retrievals (line 6), finally passes the concatenated feature into a decoder model (line 7). The time complexity for text concatenation is $O(\mathcal{G}(L_q + \sum_{i=1}^{k} L_i))$, where $L_q, L_1, \ldots, L_k$ are the sequence length of query and retrievals. If the generator is based on Transformer, then the time complexity is $O((L_q + \sum_{i=1}^{k} L_i)^2)$. The time complexity for feature concatenation is $O(k \cdot C_{\text{encode}} + \mathcal{D}_f(L_q + \sum_{i=1}^{k} L_i))$. The space complexity for text concatenation is $O(L_q + \sum_{i=1}^{k} L_i)$, for feature concatenation is $O((L_q + \sum_{i=1}^{k} L_i) \cdot d)$.

4.2 Logits-based Fusion

The logits-based fusion refers to incorporating the retrieved information into the output layers. Basically, retrieved information would be fed into the same model to obtain the logits for enhancing or calibrating the predictions. Therefore, logits-based fusion can be categorized into two branches, i.e., ensemble-based fusion and calibration-based fusion.

Algorithm 4 Logits-based Fusions.

Input: A query input q, top-k nearest neighbor knowledge $\{v_1, \ldots, v_k\}$, the generator $\mathcal{G}$.
Output: Generated response y.

1: $y_q = \mathcal{G}(q)$;
2: **for** j from 1 to k **do**
3: $\quad y_{v_j} = \mathcal{G}(v_j)$
4: **end for**
5: **if** Use ensemble **then**
6: $\quad y = \lambda \sum_j y_{v_j} + (1 - \lambda) y_q$;
7: **else**
8: $\quad \lambda_t = Calibrate(y_q, y_{v_1}, \ldots, y_{v_k})$
9: $\quad y = \lambda_t \sum_j y_{v_j} + (1 - \lambda_t) y_q$;
10: **end if**
11: **return** y;

Ensemble-based fusion treats the logits from the retrieved information as part of an ensemble of predictions. Such ensemble-based fusion can significantly improve the generalization and robustness of the model (Xiong et al. 2023; Khandelwal et al. 2020, 2021). One notable work of ensemble-based fusion is kNN-LM (Khandelwal et al. 2020), which aggregates the logits of the top-k nearest neighbors' targets and then interpolates the final predictions. Similar to kNN-LM, Khandelwal et al. (Khandelwal et al. 2021) propose kNN-MT to enhance the machine translation using retrievals' logits, which is also followed by a branch of works (Zheng et al. 2021; Huang et al. 2023c).

Different from ensemble-based fusion, calibration-based fusion uses the logits from the retrieved information as a form of calibration for the model's predictions. Specifically, Jiang et al. (Jiang et al. 2022) propose a confidence-enhanced kNN-MT that refines the kNN distribution and interpolation weights with the neural machine translation confidence. Li et al. (Li et al. 2023c) propose to leverage the source context to calibrate the retrieval-augmented neural machine translation.

Algorithm 4 demonstrates the detailed steps of using the logits-based fusion to integrate the retrieved information. This branch of work first treats retrievals as similar data to augment the model (lines 2-4). For ensemble, algorithm 4 leverages a hyperparameter to fuse the retrieval logits and the output logits (line 6). For calibration, algorithm 4 dynamically determines the parameter based on the retrieval logits and the output logits (line 8). Then, algorithm 4 performs the same fusion with the computed parameter (line 9). The time complexity is $O((k+1) \cdot \mathcal{G}(L))$ and the space complexity is $O((k+1) \cdot V)$, where L represents any sequence length of query input or retrievals, V is the vocabulary size.

4.3 Latent Fusion

The latent fusion investigates merging the retrieved information into the hidden states of generators for a better generation. Based on the introduction method, latent fusion can be further classified into two categories: attention-based and weighted-addition.

Algorithm 5 Latent Fusions.

Input: A query input q, top-k nearest neighbors $\{v_1, \ldots, v_k\}$, the encoder $\mathcal{E}$, the generator $\mathcal{G}$ containing l pairs of modules $\{(\mathcal{M}_1^A, \mathcal{M}_1^F), \ldots\}$, where $\mathcal{M}_i^A$ and $\mathcal{M}_i^F$ are the attention module and the FFN module at layer i, $\mathcal{M}_i^C$ is the cross-attention module used in attention-based latent fusions.

Output: Generated response y.

if Use the attention **then**
  $h_0^F = q$;
  for i from 1 to l **do**
    $h_i^A = \mathcal{M}_i^A(h_{i-1}^F)$;
    $e_{v_1}, \ldots, e_{v_k} = \mathcal{E}(v_1, \ldots, v_k, h_i^A)$
    $h_i^R = \mathcal{M}_i^C(h_i^A, e_{v_1}, \ldots, e_{v_k})$; /* Cross-attention */
    $h_i^F = \mathcal{M}_i^F(h_i^R)$
  end for
  $y = LM\_HEAD(h_l^F)$
else
  $e_{v_1}, \ldots, e_{v_k} = \mathcal{E}(v_1, \ldots, v_k)$
  $h_0^F = q$;
  for i from 1 to l **do**
    $h_i^A = \mathcal{M}_i^A(h_{i-1}^F)$;
    $h_i^R = h_i^A + \frac{1}{k}\sum_j w_j e_{v_j}$ /* Weighted sum */
    $h_i^F = \mathcal{M}_i^F(h_i^R)$
  end for
  $y = LM\_HEAD(h_l^F)$
end if
return y;

One notable contribution of attention-based fusion is the RETRO (Borgeaud et al. 2022). RETRO represents a pioneering effort in pre-training retrieval-based LLMs, introducing a new cross-attention module to integrate retrieved information directly into the model's hidden states. A significant finding from this work is demonstrating a scaling law for the retrieval database, where RETRO, with a 2 trillion token database, attains performance comparable to that of major models like GPT-3 and Jurassic-1, albeit with 25 times fewer parameters. Customizing the transformer model in RETRO highlights the potential of pre-trained, retrieval-enhanced architectures in improving the efficiency and scalability of LLMs.

In addition to RETRO, other studies (Cai et al. 2021; Wu et al. 2022; de Jong et al. 2022; Li et al. 2022b; Wang et al. 2023e) have contributed to the field by leveraging new attention modules to introduce external knowledge. Typically, Li et al. (Li et al. 2022b) have extended the RETRO model by decoupling the context encoding from the model inference. Wu et al. (Wu et al. 2022), Wang et al. (Wang et al. 2023b) store the hidden attention keys and values into external memory and retrieve the knowledge from the memory using an attention mechanism.

Due to the high complexity of the attention mechanism, another branch of work adopts lightweight (weighted) additions to introduce retrieved information. Fevry et al. (Févry et al. 2020) propose the EAE model that retrieves top-k related entities'

Algorithm 6 Parametric Fusions.

Input: A query input q, top-k nearest neighbors $\{v_1, \ldots, v_k\}$, the generator $\mathcal{G}$ with weight w.

Output: Generated response y.

1: Get the top-k LoRA-style modules $\{\Delta w_1, \ldots, \Delta w_k\}$ based on $\{v_1, \ldots, v_k\}$;
2: Merge those LoRA modules into one module $\Delta w = f(\Delta w_1, \ldots, \Delta w_k)$;
3: Update the LoRA modules into the generator $\hat{w} = w + \Delta w$;
4: $y = \mathcal{G}_{\hat{w}}(q)$;
5: Remove the LoRA modules for the next query $w = \hat{w} - \Delta w$;
6: **return** y;

embeddings from a learnable external memory and adds entities' embeddings to the hidden states of the model. Wu et al. (Wu et al. 2024) propose ReFusion, which explores various learnable reranking schemes to first re-weight the embeddings of the retrieved information and then use weighted addition to incorporate them into the hidden states of the model. Those approaches signify a growing trend towards models that dynamically select and integrate relevant information, paving the way for more sophisticated and nuanced language generation and understanding.

Algorithm 5 shows the steps of using latent fusion to introduce the retrieved information into the hidden states of the generator. For attention-based latent fusion, algorithm 5 first encodes the retrievals with the output states of the attention module (line 5), then uses a cross-attention module to fuse the retrieval features into the hidden state (line 6). Different from attention-based latent fusion, weighted-addition-based latent fusion adopts a more lightweight way to incorporate retrieved information (lines 10-19). Algorithm 5 first encodes the retrievals before feeding them into the generator (line 11), which can be done offline and directly stored as values in the knowledge base. Then, algorithm 5 learns a set of weights to add the retrieval features on the hidden states of generators (line 15). The time complexity for cross-attention module is similar to that of the attention module, but for weighted addition module is $O(k \cdot d)$. The space complexity for both module is similar, i.e., $O(k \cdot d)$, storing the feature vectors of retrievals.

4.4 Parametric Fusion

Parametric fusion refers to injecting retrieved information into the parameters of the generator, rather than fusing them in hidden states. The core idea is to represent each document (or passage) as a lightweight parametric representation (e.g., LoRA style update (Hu et al. 2022; Dettmers et al. 2023)) that can be plugged into the generator's feed-forward networks (FFN). In practice, this paradigm usually involves an offline stage that converts documents into parameter-efficient "knowledge modules", and an online stage that follows a Retrieve–Update–Generate workflow: the system retrieves top-k relevant documents, merges (or composes) their corresponding parameter updates to obtain a temporary adapted model, and then generates answers using the updated parameters.

Several recent works explore this direction with different design choices on how to obtain and fuse document-level parameters. PRAG (Su et al. 2025a) is a representative

framework that parameterizes each document into LoRA-style updates offline and, at inference, merges the retrieved document-specific updates into a single plug-in module for answering. DyPRAG (Tan et al. 2025) replaces per-document LoRA modules with a lightweight parameter translator that maps documents to parametric updates on the fly. This enables test-time dynamic generation of LoRA modules and reduces the storage costs of parametric retrievals. Poly-PRAG (Su et al. 2025b) shifts from the one-document–one-adapter paradigm to a many-to-few encoding. Poly-PRAG only learns a limited number of LoRA adapters and uses a document-level routing mechanism to select adapters for each document; Then, the activated adapters are merged by routing weights. This can further improve scalability and storage costs. Such parametric fusions don't modify the architecture of base generators and maintain the same efficiency as the original inference.

Algorithm 6 demonstrates a generic parametric fusion pipeline. Given a query, the system retrieves top-k documents, then obtains their parametric representations either by loading precomputed adapters (e.g., PRAG) or by translating documents into adapters on-the-fly (e.g., DyPRAG/PolyPRAG) (Line 1). Then, the system performs parameter merging, such as summation, routing-weighted merging, or orthogonal merging, to obtain the final LoRA updates (Line 2). Next, the system merges the LoRA weights to the generator weights and generate the final answer (Lines 3-4). Finally, the system recovers the model weights for the next query (Line 5). Since after merging, the inference process is exactly the same as that of the original LLMs, the time complexity is the same. The space complexity is $O(N_w \cdot l \cdot d \cdot r)$, where N_w is the number of weight matrices and r is the rank of LoRA.

4.5 Comparison of Different Retrieval Fusions

To facilitate a systematic comparison, we summarize the key characteristics of the retrieval fusions discussed above in Table 2, covering their accessibility, memory footprint, latency, and typical application scenarios. Each fusion method exhibits distinct advantages and limitations, and the choice of method depends on the specific deployment constraints and task requirements.

Accessibility. Only query-based fusion (text concatenation) and logits-based fusion are compatible with black-box LLMs, as they only require input text or output logits, both commonly exposed via APIs. In contrast, other fusions demand white-box access, requiring architectural modifications (e.g., cross-attention modules) or parameter injection (e.g., LoRA adapters). Those retrieval fusions usually require fine-tuning or even pre-training the LLMs.

Memory and Latency. Parametric fusion achieves the highest memory and inference efficiency: after merging the retrieved LoRA module, the model behaves exactly like the base LLM, with all knowledge embedded in the weights, incurring no extra cost. Latent fusion also exhibits low memory overhead—it only stores compact representations of retrieved information (e.g., k d-dimensional vectors, where d is smaller than the LLM's hidden size). However, attention-based latent fusion introduces moderate latency due to cross-attention computation, while weighted-addition variants remain efficient. Logits-based methods strike a middle ground: they require running the LLM k times (once per neighbor) to obtain logits, which can be batched to reduce

Table 2 Comparison of different retrieval fusions.

Retrieval Fusions	Representative Works	Accessibility	Memory	Latency	Impl. Compl.	Scenarios
Query-based (Text Concat)	REALM, RAG, REINA, RALM	Black-box, White-box	High	High	Low	Rapid deployment, frequently updated knowledge, proprietary LLMs
Query-based (Feature Concat)	FID, RETRO-PROMPT, LUMEN	White-box (require fine-tuning decoders)	High	Moderate	Moderate	Knowledge-intensive tasks, encoder-decoder architecture
Logits-based (Ensemble)	kNN-LM, kNN-MT, kNN-Adapter	Black-box, White-box (require logits)	Moderate	Moderate	Moderate	Lightweight integration, plug-and-play
Logits-based (Calibration)	Robust-kNN-MT, Source-Context	Black-box, White-box (require logits)	Moderate	Moderate	Moderate	Robust generation, uncertainty calibration, domain adaptation
Latent (Attention)	RETRO, Enc-Dec, LONGMEM	White-box (require architecture modification)	Low	Moderate	High	Large-scale pre-training, complex reasoning tasks
Latent (Weighted Addition)	EAE, ReFusion	White-box (require architecture modification)	Low	Low	High	Entity enhancement, lightweight injection, and friendly fine-tuning
Parametric	PRAG, DyPRAG, Poly-PRAG	White-box (require parameters injection)	Low	Low	High	Static knowledge, domain knowledge adaptation

latency at the cost of increased peak memory. Query-based methods are the least efficient; text concatenation dramatically lengthens the input sequence, inflating the key-value (KV) cache and attention time. Feature concatenation improves throughput by encoding each retrieval independently, but still suffers from high memory usage due to long sequence length after the concatenation.

Implementation Complexity. Query-based text concatenation is the simplest (low complexity), as it operates entirely outside the model, i.e., retrieved texts are merely prepended to the input prompt, requiring no knowledge of the LLM's internal architecture. Feature concatenation and logits-based methods (both ensemble and calibration) entail moderate complexity: they interact with the model only at specific interfaces, e.g., after the encoder or at the output layer, without needing to modify internal layer structures. In contrast, latent fusion (attention and weighted addition) and parametric fusion demand high implementation complexity. These methods

require deep architectural understanding, often involving modifications such as inserting cross-attention modules, introducing learnable reranking mechanisms, or injecting LoRA adapters. Moreover, they typically necessitate fine-tuning or even pre-training to achieve effective integration, making them suitable only for scenarios where full model access and substantial development resources are available.

Applicable Scenarios. Query-based fusion with text concatenation enables rapid deployment with frequently updated knowledge, especially suitable for proprietary LLMs. Feature concatenation targets knowledge-intensive tasks, specifically for encoder-decoder architectures. Logits-based ensemble methods offer plug-and-play integration, while calibrated variants enhance robustness under retrieval noise. attention-based latent fusion is particularly suitable for complex reasoning and pre-training; weighted addition provides lightweight injection for entity-centric tasks. Parametric fusion is best for static domain knowledge, encoding expertise into reusable adapters.

4.6 Hybrid Retrieval Fusion

Beyond using a single fusion strategy, recent works have explored hybrid fusions that combine multiple retrieval fusions to leverage their complementary strengths. The most common paradigm is to use query-based fusion to inject retrieved information into the prompt, and then augment the inference with another fusion strategy (e.g., logits-based fusion, latent fusion, or parametric fusion) to improve robustness and information utilization. Text concatenation is often chosen as the "base" because it only modifies the input without requiring access to hidden states/logits or additional model training. For example, DVD (Jin et al. 2024b) combines query-based fusion (text concatenation) and logits-based fusion (ensemble) to improve the generation quality. And PRAG (Su et al. 2025a) proposes to use the parametric fusion to inject knowledge, but further improves performance by integrating the query-based fusion (text concatenation).

However, hybrid fusion is not limited to text concatenation. The text concatenation always introduces high computational and memory overheads due to the long input sequence. Therefore, exploring other combinations of different retrieval fusions becomes a necessity. For example, integrating logits-based fusion with latent fusion can inject the knowledge into models via lightweight modules, without increasing input sequence length and enabling deep contextual reasoning. With the development, we anticipate a richer landscape of hybrid fusions, where different strategies are adaptively selected or blended based on task requirements and resource constraints.

5 Generators

In a RAG system, the *generator* is the conditional language model that maps the user query q and retrieved information $Z_q = \{z_i\}_{i=1}^{k}$ to an output sequence y. Compared to a standalone LLM, a RAG generator must not only be fluent, but also (i) consume longer and noisier contexts, (ii) use retrieved information instead of relying on parametric memory, and (iii) ideally ground responses with verifiable attribution. Most modern generators are Transformer-based models (Vaswani et al. 2017)

and are dominated by decoder-only LLMs such as GPT-series (Radford et al. 2018, 2019; Brown et al. 2020; OpenAI 2023), Llama-series (Touvron et al. 2023a,b; Team 2024), Gemini/Gemma (Anil et al. 2023; Reid et al. 2024; Mesnard et al. 2024), DeepSeek (DeepSeek-AI 2025), Qwen series (Yang et al. 2024a,b), and Mistral/Mixtral (Jiang et al. 2023a, 2024a). These generators typically incorporate architectural optimizations such as RMSNorm (Zhang and Sennrich 2019), rotary position embedding (RoPE) (Su et al. 2024a), and grouped-query attention (GQA) (Ainslie et al. 2023), which are especially relevant in RAG because retrieval may increase the input length and amplify inference-time memory/latency costs.

5.1 Architectural Requirements for Effective RAG

A generator is "RAG-effective" only when its architecture and training make it both capable of ingesting retrieved information and willing to use it. We summarize key requirements and their architectural implications:

- **Information Capacity and Context Management.** Query-based fusion (e.g., text concatenation that concatenates retrieved information into the prompt) requires large context windows and efficient use of attention/KV-cache usage. Recent LLMs extend context length substantially (e.g., Llama 3 up to 128K (Team 2024), DeepSeek-V2 supports 128K (DeepSeek-AI 2025), Gemini 1.5 reports million-token-scale context (Reid et al. 2024)). However, long context does not guarantee robust information usage. Models can be position-sensitive and degrade when relevant information is "lost in the middle" (Liu et al. 2024). Therefore, generator design must consider not only supporting long-context generation but also effective context utilization under long inputs.
- **Utilizing Information beyond Simple Concatenation.** When substantial information is retrieved, naïve query-based fusion often causes attention dilution and budget waste. Architectures that *separate information encoding from generation* can scale better. RETRO (Borgeaud et al. 2022) modifies the architecture of naive Transformer blocks by introducing a cross-attention module between the self-attention module and FFN module to inject the encoded information into the model. This line of works (Li et al. 2022b; Borgeaud et al. 2022) exhibit extremely strong language modeling capability by pre-training the model. ReFusion (Wu et al. 2024) also proposes to introduce some lightweight modules and fine-tunes the modified model for robustness. Those works demonstrate that architectural fusions can further improve the models' performance but at the cost of training.
- **Grounding, Attribution, and Controllability.** RAG applications often require citations, abstention when information is missing, and controllable behaviors (e.g., "answer-only-from-information"). This typically needs either retrieval-aware training objectives or explicit control tokens and supervision. For instance, Self-RAG trains a single LM to retrieve on-demand and to generate/refine with special reflection tokens, improving factuality and citation behavior (Asai et al. 2024).
- **Robustness to Noisy/Conflicting Retrieval.** Retrieval quality is sometimes imperfect; retrieved information can be irrelevant or contradictory. Generators

benefit from mechanisms that reduce over-reliance on any single passage, such as document-level marginalization (as in RAG's latent document variable) (Lewis et al. 2020), filter unrelated retrieval (Chen et al. 2026), or training-time exposure to mixed-quality retrieval (e.g., Atlas, Self-RAG) (Izacard et al. 2023; Asai et al. 2024).

- **Efficiency under Retrieval-Augmented Inference.** RAG often increases latency due to the retrieval and the increasing generation cost with longer prompts. Thus, inference-optimized attention designs are practically important: GQA reduces KV-cache overhead while preserving quality (Ainslie et al. 2023); DeepSeek-V2 compresses KV cache via Multi-head Latent Attention (MLA) and adopts MoE for economical scaling (DeepSeek-AI 2025).

5.2 Generator Types

From an architectural and deployment standpoint, generators in RAG can be categorized more informatively than "open vs. closed":

- **Proprietary/API LLMs.** Examples include GPT-4 (OpenAI 2023) and Gemini (Anil et al. 2023; Reid et al. 2024). They are attractive for strong general capability and engineering maturity, but usually constrain RAG to prompt-time fusion (concatenation, instruction templates, and multi-call orchestration). The key limitation is architectural immutability. One cannot insert cross-attention information modules or do retrieval-aware pretraining. Nevertheless, black-box RAG can still improve by optimizing retrieval and prompt composition. RePlug shows that one can treat the LLM as frozen and tune the retriever using LLM signals, improving black-box LLMs without changing generator weights (Shi et al. 2024).
- **Open-Weight Decoder-only LLMs.** Models such as Llama (Touvron et al. 2023a,b; Team 2024), DeepSeek (DeepSeek-AI 2025), enable system designers to: (i) finetune for domain style and grounding, (ii) add adapters (e.g., LoRA) or tool-use interfaces, and (iii) experiment with retrieval-aware training curricula (e.g., instruction tuning with retrieved information). The trade-off is that model quality, safety alignment, and multilingual robustness may lag proprietary systems at equal compute budgets, and finetuning introduces data/compute requirements.
- **Retrieval-Native/Retrieval-Aware Generators.** These models explicitly integrate retrieval into model design and/or pretraining. Representative families include: (a) seq2seq RAG generators such as RAG (Lewis et al. 2020) and FiD (Izacard and Grave 2021), (b) retrieval-augmented pretraining such as REALM (Guu et al. 2020) and Atlas (Izacard et al. 2023), (c) retrieval-enhanced decoder-only designs such as RETRO with chunked cross-attention over retrieved neighbors (Borgeaud et al. 2022), and (d) non-parametric memory augmentation such as kNN-LM (Khandelwal et al. 2020). These approaches typically yield stronger information usage and easier knowledge updating by swapping the index, but require more complex training/inference pipelines and careful retriever–generator co-design.

5.3 How does Generator Design Impact RAG Performance

Generator architecture shapes RAG performance along several axes: **(i) Information sensitivity:** cross-attention or separate information encoding (FiD/RETRO) tends to use information more systematically than raw concatenation (Izacard and Grave 2021; Borgeaud et al. 2022). **(ii) Scaling with top-k:** models that can amortize information encoding and avoid full self-attention over all retrieved information tokens can scale to larger k, which is empirically beneficial in multi-document QA (Izacard and Grave 2021). **(iii) Long-context reliability:** long-context LLMs enable larger retrieved information budgets, but may still fail to access mid-context retrieved information, motivating ordering strategies and architectural fusion for robustness (Liu et al. 2024). **(iv) Grounding behavior:** retrieval-aware training (Atlas (Izacard et al. 2023), Self-RAG (Asai et al. 2024)) improves factuality behavior and reduces hallucination relative to prompt-only RAG under similar retrieval settings.

5.4 Practical Guidelines for Selecting Generators

Based on the above trade-offs, we provide selection guidelines according to different situations:

- **Generator weights are inaccessible (API-only):** prioritize strong retriever/reranker, compact information formatting, and multi-stage prompting. Methods that tune retrieval while keeping the LLM frozen are suitable when you cannot change the generator (Shi et al. 2024; Chen et al. 2025c). Long-context models help, but retrieved information should be positioned carefully due to long-context utilization issues (Liu et al. 2024).
- **Need controllable grounding/citations or domain compliance:** choose open-weight generators and apply retrieval-aware finetuning (instruction tuning with retrieved information, contrastive supervision for citation, etc.), or use explicitly retrieval-aware generators (Atlas (Izacard et al. 2023), Self-RAG (Asai et al. 2024)) when training resources allow.
- **Task needs many passages (multi-hop QA, long-form synthesis):** prefer architectures designed for multi-document fusion (FiD-style encoder-decoder or retrieval-native cross-attention) rather than prompt stuffing, because they scale better with top-k and reduce attention dilution (Izacard and Grave 2021; Borgeaud et al. 2022).
- **Latency/cost is critical:** select generators with inference-efficient attention or memory designs (e.g., GQA, sliding-window attention, KV compression, MoE) and keep retrieved context concise (Ainslie et al. 2023; DeepSeek-AI 2025).
- **Knowledge updates are frequent:** retrieval-native/aware paradigms that separate parametric knowledge from an updatable index are preferable (RAG/REALM/Atlas/RETRO/kNN-style memory), since updating the corpus/index can be cheaper than continual finetuning (Lewis et al. 2020; Guu et al. 2020; Izacard et al. 2023; Borgeaud et al. 2022; Khandelwal et al. 2020).

Overall, generator choice is not merely a matter of "which LLM is strongest", but of how the generator's architecture and training interface with retrieval, which directly determines information utilization, grounding quality, and system-level efficiency.

6 NLP Tasks

This section lists several classical tasks in the NLP domain, introduces advanced RAG techniques used to solve these tasks, and discusses the task-specifc challenges and failure modes

6.1 Language Modeling

Task Characteristics. Language modeling refers to the capability of modeling the probability distribution over sequences of tokens. In practice, this capability is instantiated as the next-token prediction task: given such a sequence of tokens $x_1, \ldots, x_n$ called *Prefix*, the language modeling task aims to model its probability via next-token prediction,

$$p(x_1, \ldots, x_n) = p(x_1) \cdot \prod_{i=2}^{n} p(x_i | x_1, \ldots, x_{i-1}), \tag{3}$$

where the conditional probabilities $p(x_i|x_1, \ldots, x_{i-1})$ are modeled by a parameterized language model.

Technique Summary. Recent works mainly leverage RAG further to improve language modeling capability in the pre-training stage. A branch of works (Borgeaud et al. 2022; Li et al. 2022b; Wu et al. 2022; Wang et al. 2023b) modifies the architecture of generators by adding a new cross-attention module in each transformer block for introducing retrieval knowledge. The intuition of those works is that given the similar *Prefix*es and their next tokens (retrieving stage), the pre-trained model can calibrate the model's prediction using the cross-attention module to capture the pattern between the next token and prefix (model forwarding stage). Zhong et al. (Zhong et al. 2022) propose to augment the language model with three types of retrieval memories/-databases (local memory, long-term memory, and external memory) and optimize the next-token probability distribution with nearest neighbors retrieved from the memories/databases. Another branch of works (Khandelwal et al. 2020; Huang et al. 2023c; Xu et al. 2023; Ram et al. 2023b; Guu et al. 2020) focuses on augmenting the inputs or outputs of generators with retrievals. Guu et al. (Guu et al. 2020) and Ram et al. (Ram et al. 2023b) concatenate the retrieved information with inputs and feed the retrieval-augmented inputs into the generators. Other works (Khandelwal et al. 2020; Huang et al. 2023c; Xu et al. 2023) fuse the logits of inputs as well as retrievals at the final output layer and generate the final probability distribution based on the interpolated results. Those works believe that the concatenated/fused retrievals can provide useful context information on inputs/outputs to improve models' robustness during the pre-training stage. Besides, Doostmohammadi et al. (Doostmohammadi et al. 2023) focus on pre-training models with a semantic retriever (BM25) and achieve a better language modeling performance.

Challenges and Failure Modes. A main challenge in retrieval-augmented language modeling is that the performance is highly sensitive to the retrieval quality. This

is because language modeling is evaluated by perplexity, a metric sensitive to every single token prediction. Even minor misalignment between the retrieved next token and the true next token would directly harm the performance, and the retrieval noise will be amplified rather than diluted. Consequently, common failure modes can be: (1) spurious copying of neighbors' next token that do not truly match the true next token, (2) noise amplification that shifts probability mass toward incorrect tokens and increases repetition or incoherence.

6.2 Machine Translation

Task Characteristics. Machine translation (MT) leverages computational linguistics algorithms to translate text or speech from one language to another automatically. The goal of MT is to produce an accurate and fluent translation, preserving the meaning of the original text while adhering to the grammatical and stylistic norms of the target language. MT systems have evolved from rule-based machine translation (RBMT) to statistical machine translation (SMT) and, more recently, to neural machine translation (NMT). In particular, NMT methods have significantly improved translation quality by leveraging deep learning techniques, which thus will be the focus of this section.

Technique Summary. RAG techniques can further enhance MT by incorporating external knowledge into the translation process. The simplest way is to concatenate the similar translation examples into the inputs or fuse the logits of similar translation examples at the output layer. For example, some works (Wang et al. 2022; Cheng et al. 2023b) retrieve similar translations according to the source text and concatenate corresponding target texts or pairs of source and target texts as examples into inputs. Other works (Hossain et al. 2020; Khandelwal et al. 2021; Zheng et al. 2021) feed the retrieved source text into the models and obtain the logits of the next target tokens, then aggregate all logits to generate the final predictions. Moreover, Jiang et al. (Jiang et al. 2022) and Li et al. (Li et al. 2023c) use the logits of retrieved examples to calibrate the aggregated logits, improving the robustness of the generation. Another branch of works (Zhu et al. 2023; Zhong et al. 2022) injects external knowledge into the objective function during the training stage, refining the representation space with similar translations. Besides, Cai et al. (Cai et al. 2021) encode similar translations and store them as the translation memory, then introduce the knowledge from memory with a cross-attention module. Instead of improving the performance, a branch of work focuses on accelerating the generation efficiency on MT tasks, such as searching from a pre-built subset (Meng et al. 2022b; Deguchi et al. 2023) or a dynamic knowledge base (Dai et al. 2023), searching by chunks (Martins et al. 2022).

Challenges and Failure Modes. In MT, retrieval augmentation must improve adequacy while preserving fluency and terminology consistency, which requires retrieving examples that are not only source-side similar but also reliable and stylistically compatible on the target side. However, mismatched examples can introduce subtle lexical/syntactic biases, and document-level consistency (e.g., domain terms and named entities) may not be guaranteed by sentence-level retrieval. Accordingly, failure modes often appear as: (1) retrieval-biased terminology drift (incorrect or inconsistent

term transfer), (2) over-copying retrieved target fragments leading to entity/number errors and reduced adequacy.

6.3 Text Summarization

Task Characteristics. Text summarization is the process of condensing a larger text document into a shorter version, preserving key information and the overall message. This task can be broadly categorized into two types: extractive summarization, which involves selecting and compiling parts of the original text, and abstractive summarization, which entails rewriting the essence of the text in a new, concise form. The goal is to produce a coherent and fluent summary that encapsulates the most critical information from the source material.

Technique Summary. RAG techniques can significantly enhance text summarization tasks by leveraging external knowledge and similar documents to inform the summarization process. (Wang et al. 2022; Fan and Gardent 2022; Li et al. 2023d; Cheng et al. 2023b) simply concatenates the retrieved similar summaries into inputs to generate summarizations. Instead of concatenating texts, other works fuse features at the intermediate layers by cross-attention (Bertsch et al. 2023), or at the output layers by logits ensemble (Hossain et al. 2020). Besides, Jiang et al. (Jiang et al. 2023b) argue that retrieving for every generation may not always be the best choice and propose to retrieve external knowledge during the generation process adaptively.

Challenges and Failure Modes. For summarization, the central challenge is balancing compression with grounding: retrieval can supply helpful external information, but it also tends to introduce redundancy and tangential details that compete with the source document for attention under a limited context budget. Furthermore, summarization often requires abstraction and synthesis rather than copying, so naïvely injecting retrievals can bias the model toward extractive behavior. As a result, typical failure modes include (1) content contamination, where irrelevant retrieved facts leak into the summary and reduce faithfulness, (2) retrieval accuracy, where the summarization task often requires document-level retrievals which may occur semantic drift issues, and (iii) copy bias that harms abstraction and can even import contradictions from noisy retrievals.

6.4 Question Answering

Task Characteristics. Question Answering (QA) is a fundamental task in NLP that involves building systems capable of automatically answering human questions in natural language. QA systems can be broadly classified into two categories: open-domain, where the system answers questions about virtually anything, and closed-domain, focusing on a specific area of knowledge. Due to the page limits, this paper only discusses the works of open-domain QA systems.

Technique Summary. RAG techniques combine information retrieval with model-based generation, which is highly suitable for QA systems. In particular, open-domain QA systems usually first require searching for knowledge from the Internet or large-scale databases, then generate the corresponding answers according to the retrieved information. Naturally, given similar questions and corresponding answers as

demonstrations which are concatenated into inputs (Wang et al. 2022; Li et al. 2023d; Huang et al. 2023a), generators in RAG can learn the pattern between questions and answers and infer what answers should be. For some specific QA tasks where a set of reference documents is given, retrievers in RAG would retrieve the relevant documents for concatenation, and then generators in RAG would read the context then generate the final answers via the self-attention mechanism (Guu et al. 2020; Lee et al. 2023; Ram et al. 2023b; Asai et al. 2024), which is similar to solving a reading comprehension problem. Besides, Fabbri et al. (Fabbri et al. 2020) focus on designing effective templates for re-organizing the concatenated contexts. Baek et al. (Baek et al. 2023) leverage the knowledge graph to retrieve the related facts for the input questions, then feed their concatenation and inputs into the generators. Instead of directly concatenating texts, another branch of works focuses on joining the retrieval embeddings with input embeddings for the encoder-decoder models (Izacard and Grave 2021; Sachan et al. 2021; de Jong et al. 2023b; Izacard et al. 2023).

Some works incorporate the external knowledge in the hidden states or the final logits of generators. For the fusion in the hidden states, the key is what kind of knowledge representation would be injected, such as entities (Févry et al. 2020; de Jong et al. 2022), chunks (Borgeaud et al. 2022; Wang et al. 2023a), documents (Cheng et al. 2023a). For the fusion in the logits, most works combine the logits of retrievals and inputs by ensemble techniques (Shi et al. 2024; Guu et al. 2020; Lewis et al. 2020; Mueller et al. 2023).

Instead of designing different knowledge fusions for QA systems, existing works also improve QA systems with RAG from other aspects. Some works (Guo et al. 2019; Paranjape et al. 2022; Lin et al. 2022) use retrieved question-answering pairs as extra training data. Some works optimize the retriever module, e.g., improving the keys' representation when building the retriever database (Ram et al. 2023a), replacing the indexing with a pre-trained ranking model (Yu et al. 2023b), or enabling retrieving phrases with two queries (Min et al. 2023b). Other works focus on accelerating the generation efficiency of RAG. Jong et al. (de Jong et al. 2023a) propose the layer-sparse cross-attention to speed up the decoding. Some works (Asai et al. 2024; Jiang et al. 2023b; Wang et al. 2023c) observe that the retrievals may not always provide useful information during the generation process and learn to determine when to retrieve. Moreover, Sun et al. (Sun et al. 2024) combine the RAG with agents to iteratively reason the final results.

Challenges and Failure Modes. Open-domain QA is highly sensitive to retrieval quality because those tasks require the retrieved passages to contain the exact answer span. In addition, QA systems are frequently expected to provide verifiable grounding (e.g., attribution/citations) and to abstain when retrieved information is insufficient, which further raises the bar for information utilization. Accordingly, failure modes commonly manifest as: (1) missing-information hallucination, where the model answers from parametric memory when the retriever fails to obtain the passage that contains answers, (2) cherry-picking under contradiction, where the model selectively uses one snippet while ignoring conflicting retrieved information.

6.5 Information Extraction

Task Characteristics. Information Extraction (IE) is a critical task in NLP to automatically extract structured information from unstructured and semi-structured text sources. This task encompasses several sub-tasks, including Named Entity Recognition (NER), Entity Linking (EL), Coreference Resolution (CR), Relation Extraction (RE), etc. The goal is to identify and classify key elements from text and understand the relationships between them, thereby converting textual data into a structured format amenable to analysis and interpretation.

Technique Summary. With RAG techniques, addressing IE tasks can be significantly improved in terms of not only performance but also interpretability. In NER tasks, Wang et al. (Wang et al. 2021b) first retrieve similar sentences and then concatenate the ranked retrievals for better semantic representations. Ren et al. (Ren et al. 2023) show that naive RAG may not address Event Argument Extraction (EAE) tasks. Thus, they adopt a sampling-based method to guarantee the same distribution of event labels between retrievals and inputs then concatenate retrieval texts into inputs for better performance in EAE tasks. Table augmentation is also a challenging task, which requires extracting information from tables. Glass et al. (Glass et al. 2023) propose to leverage a reconstruction-based method with a trained retrieval-based model to address table augmentation.

Challenges and Failure Modes. The challenge in applying RAG to information extraction lies in how to effectively integrate structured external knowledge, such as knowledge graphs for entities and relations or table data. Simple concatenation fails to fully convey the relational and type information inherent in such structured data, necessitating more sophisticated fusion mechanisms. Besides, useful retrievals must be not only semantically similar but also label-compatible (e.g., entity types, relation schemas, event templates), otherwise the retrieved patterns can mislead span boundaries and role assignments. Consequently, failure modes often appear as: (1) information injection failure, where those structured information cannot be effectively injected into the model for accurately extraction, (2) schema/label drift, where retrievals reflecting different conventions degrade extraction consistency.

6.6 Text Classification

Task Characteristics. Text classification tasks are common in NLP applications (Siino et al. 2025). Sentiment analysis, a prominent text classification task in NLP, entails identifying and categorizing the emotional tone conveyed in a text. For example, given a sentence of "I love to watch movies", the analysis models should determine whether it has a positive attitude or a negative attitude. The attitude in sentiment analysis can range from positive to negative or can be neutral, nuanced, and even mixed. The sentiment analysis task is crucial for understanding consumer feedback, monitoring brand reputation, and gaining insights into public opinion on various issues.

Technique Summary. RAG techniques can significantly enhance sentiment analysis with different external knowledge fusion strategies. Li et al. (Li et al. 2023d) concatenate the retrieved options and corresponding prompt-based labels with

input options. Other works (Chen et al. 2022b; Guo et al. 2023) concatenate the retrieval embeddings with input embeddings before feeding them into the decoder. Some works fuse the retrieval features into the hidden states of generators via cross-attention (Cheng et al. 2023a; Wang et al. 2023b) or ranking-based addition (Wu et al. 2024). Besides, other works focus on fusing the logits of retrievals with the output logit using ensemble techniques (Zhang et al. 2023b; Yu et al. 2023a). Except for knowledge fusions, Min et al. (Min et al. 2023b) enable locating knowledge in phrases more accurately via two queries.

Challenges and Failure Modes. For text classification, retrieval noise can directly mislead the classification, as even semantically similar documents may carry different labels. Moreover, the problem is exacerbated under domain shift, where retrieval may reinforce dataset artifacts rather than true label semantics. Accordingly, failure modes typically manifest as: (1) nearest-neighbor confusion, where retrieved instances are similar but differently labeled and systematically bias predictions, (2) demonstration-induced bias/instability, where predictions fluctuate with changes in retrieved examples or their ordering.

6.7 Dialogue Systems

Task Characteristics. Dialogue systems, also known as conversational agents or chatbots, are designed to simulate conversation with human users, either in text or speech form. These systems can be categorized into two main types: task-oriented systems (Huang et al. 2023b), which assist users in completing specific tasks such as booking tickets or ordering food, and open-domain systems, which aim to carry on a general conversation on a wide range of topics (Shuster et al. 2021). The core challenge in developing effective dialogue systems lies in understanding user intent, maintaining context, and generating coherent, relevant responses.

Technique Summary. Existing works improve the dialogue system with RAG mostly via the query-based fusions. Some works (Li et al. 2021; King and Flanigan 2023; Cheng et al. 2023b) concatenate the retrieved history conversations with current inputs. Other works (Fan et al. 2021; Liu et al. 2023b; Cheng et al. 2023b) first leverage an encoder to encode the history responses, then feed the concatenated embeddings into a decoder to generate new responses.

Challenges and Failure Modes. Dialogue systems operate under multi-turn context accumulation, making retrieval augmentation challenging in selecting what to retrieve (history, profile, external knowledge) without overwhelming the context window or harming conversational flow, while also respecting privacy and latency requirements. In this setting, failure modes commonly manifest as: (1) stale or incorrect memory grounding, which causes inconsistencies across turns, (2) evolution of user intent, where user intent may change dynamically during the conversation.

7 RAG Evaluation and Benchmark

The evaluation of RAG systems is inherently more complex than evaluating standalone retrievers or language models. Performance hinges on the synergistic interaction

Table 3 Taxonomy of RAG Evaluation Metrics and Dimensions. The taxonomy organizes metrics by their primary evaluation focus, moving from classic retrieval to RAG-specific generation and robustness, culminating in system-level operational concerns. Note: Answer Faithfulness is distinct from Answer Correctness; an answer can be faithful to an incorrect retrieved context.

Evaluation Focus	Dimension	Metrics	Representative Works
Retrieval Quality	Context Relevance	Precision@k, Recall@k, Hit Rate, Mean Reciprocal Rank (MRR), Normalized Discounted Cumulative Gain (NDCG)	TREC, MS MARCO, BEIR (Thakur et al. 2021), Domain-specific adaptations (Pipitone and Alami 2024)
Generation Performance	Answer Faithfulness (Groundedness)	Answer Attribution / Faithfulness Score, Citation Precision/Recall	RAGAS (ES et al. 2024), ARES (Saad-Falcon et al. 2024), FActScore (Min et al. 2023a), TRUVA (Ni et al. 2025)
	Answer Relevance	Answer Similarity (BERTScore, ROUGE), LLM-as-a-Judge relevance scoring	RAGAS (ES et al. 2024), AlpacaEval (Dubois et al. 2024)
	Answer Correctness	Exact Match (EM), F1 Score, Task-Specific Accuracy	OpenQA (Siriwardhana et al. 2023), Domain-specific (e.g., (Xiong et al. 2024; Hui et al. 2024))
End-to-End Robustness	Noise Robustness, Negative Rejection, Information Integration, Counterfactual Robustness	Accuracy, Rejection Rate, Error Detection Rate, Error Correction Rate	RGB (Chen et al. 2024)
System-Level Efficiency	Operational Performance	Latency (p95, p99, TTFT), Cost (USD per query), Throughput, Memory Footprint	Refusion (Wu et al. 2024), ACIN (Wornow et al. 2024), HedraRAG (Hu et al. 2025b)

between two core components: the retriever's ability to fetch relevant, context-supporting information, and the generator's ability to faithfully and accurately synthesize this knowledge into a coherent response. Consequently, effective evaluation must disentangle these contributions while also assessing the integrated system's end-to-end efficacy. This section provides a comprehensive taxonomy of RAG-specific metrics, analyzes fundamental evaluation challenges, compares prevailing methodologies, and critically examines the limitations of existing benchmarks to illuminate paths for future work.

7.1 A Taxonomy of RAG Evaluation

Table 3 provides a consolidated overview of RAG evaluation metrics, organized by evaluation focus. We categories them into four evaluation focus, including retrieval quality, generation performance, end-to-end robustness, and system-level efficiency.

Retrieval Quality assesses the quality of the retrieved context. *Context Relevance*, the primary dimension here, encompasses traditional information retrieval metrics—such as Precision@k, Recall@k, and NDCG—that measure the topical alignment between retrieved passages and the query.

Generation Performance focuses on the final output across three distinct dimensions. *Answer Faithfulness* (or Groundedness) measures whether the generated answer is entirely supported by the provided context, serving as a critical guard against hallucinations (Park and Lee 2024; ES et al. 2024). *Answer Relevance* assesses whether the response addresses the user's query and remains on-topic, while *Answer Correctness* measures factual accuracy against a ground truth answer (Friel et al. 2024). These dimensions are conceptually distinct: an answer can be faithful to retrieved information yet factually incorrect if the information itself is wrong, underscoring the need for comprehensive evaluation across all three.

End-to-End Robustness extends beyond basic quality metrics to evaluate system behavior under challenging conditions. Following frameworks like RAG-Bench (Chen et al. 2024), this dimension encompasses several distinct "abilities": *Noise Robustness* (handling irrelevant or distracting context), *Negative Rejection* (refusing to answer when retrieved information is insufficient), *Information Integration* (synthesizing multiple retrieved information pieces), and *Counterfactual Robustness* (resisting misleading information). Recent benchmarks such as RaLLe (Hoshi et al. 2023) provide targeted datasets to stress-test these capabilities.

System-Level Efficiency captures operational considerations that determine deployability. Beyond answer quality, practical RAG systems must satisfy constraints on *latency* (particularly tail latencies such as p95 and p99, as well as time-to-first-token), *cost* (e.g., USD per query), *throughput* (queries per second under fixed service-level objectives), and *memory footprint* (including index size and cache overhead). These metrics enable bottleneck attribution and guide architectural choices. For instance, ACIN (Wornow et al. 2024) reports per-query costs, ReFusion (Wu et al. 2024) demonstrates latency reductions through architectural fusion, and HedraRAG (Hu et al. 2025b) emphasizes cross-stage co-optimization to improve end-to-end latency and throughput predictability.

7.2 Key Evaluation Challenges

The interdependency of retrieval and generation creates unique evaluation pitfalls. One key challenge is the trade-off between context faithfulness and context relevance (Ji et al. 2023a). The faithfulness–relevance trade-off highlights a fundamental tension in text generation. An answer that is strictly faithful to the provided context may still be of low quality if the context is only marginally relevant to the question. Conversely, an answer that is highly relevant to the user's intent may require reasoning beyond the

given context, thereby sacrificing faithfulness. The optimal balance between faithfulness and relevance is inherently application-dependent. For instance, legal or medical question answering prioritizes strict faithfulness to source documents (Lehman et al. 2023; Chalkidis et al. 2020), whereas creative writing or brainstorming tasks may tolerate, or even benefit from, reduced faithfulness in favor of higher relevance and usefulness.

Another significant issue is the difficulty of determining whether an error originates from the retriever or the generator. Specifically, an incorrect answer may result either from the retriever's failure to surface relevant or sufficient retrieved information, or from the generator's failure to faithfully interpret and utilize otherwise correct retrieved information (Lewis et al. 2020; Ru et al. 2024). Disentangling these two sources of error is non-trivial, as both components are tightly coupled during inference and are typically evaluated only through the final generated output (Friel et al. 2024; Ru et al. 2024). Accurate attribution is nevertheless crucial for effective system debugging, evaluation, and model improvement. Without such attribution, designing targeted interventions—such as improving retrieval quality, enhancing grounding constraints, or refining decoding strategies—becomes challenging. However, reliable attribution often requires fine-grained annotations, such as information-level relevance judgments or claim-level faithfulness labels, which are expensive and time-consuming to obtain, particularly in domain-specific settings like legal or medical question answering. As a result, many existing benchmarks and evaluations conflate retrieval and generation errors, obscuring the true source of system failures and limiting the interpretability of empirical results. (Yue et al. 2023; Li et al. 2024b)

A third challenge concerns the correlation of automatic metrics with human judgment. In open-ended RAG and other long-form generation settings, traditional lexical-overlap metrics such as BLEU and ROUGE are often unreliable proxies for human-perceived answer quality: meta-evaluation studies report weak or inconsistent correlations with human ratings on key dimensions like coherence and relevance. Overlap-based scoring can also be misleading for certain question answering or machine reading comprehension answer types (e.g., yes/no or entity-list questions) (Fabbri et al. 2021; Sellam et al. 2020). Moreover, recent RAG-specific analyses note that n-gram–based metrics may work for short answers but fail to capture fine-grained quality differences in longer, multi-claim responses, motivating more diagnostic evaluation protocols (Ru et al. 2024). These limitations have contributed to a shift toward model-based, reference-free evaluation, particularly the LLM-as-a-judge paradigm, which uses strong LLMs to grade responses under explicit rubrics and has demonstrated substantially better alignment with human preferences on open-ended tasks (Liu et al. 2023c; Zheng et al. 2023).

Evaluating unknown or refusal responses is also challenging in open-ended RAG systems. A reliable model should abstain (e.g., answer "I don't know" or refuse) when the retrieved information is missing, contradictory, or irrelevant to the query, rather than hallucinating. However, measuring this capability requires carefully constructed negative instances—such as adversarially unanswerable questions or deliberately flawed contexts—beyond standard answerable QA data (Rajpurkar et al. 2018; Sulem

et al. 2021; Muhamed et al. 2025). Moreover, the metric must avoid rewarding trivial over-refusal (e.g., refusing everything): selective prediction formulations make this trade-off explicit by jointly evaluating accuracy (risk) and the fraction o0f questions answered (coverage), for example using risk–coverage curves or coverage at a fixed risk level (Xin et al. 2021).

7.3 Comparison of Evaluation Methodologies

RAG systems can be evaluated using a spectrum of methodologies, each trading off validity, cost, and scalability. In practice, robust assessment often combines multiple methods rather than relying on a single score.

Human Evaluation: The traditional gold standard, where trained annotators rate dimensions such as fluency, factual correctness, relevance, and faithfulness/-grounding to the retrieved information. When carefully designed with clear rubrics and quality control, it offers high validity and diagnostic value. However, it is expensive, slow, and difficult to scale, making it impractical for rapid iteration or large-scale benchmarking.

Automatic Metric-Based Evaluation: Relies on predefined, programmatic metrics (e.g., EM, token-level F1, BERTScore) for fast and reproducible scoring (Siriwardhana et al. 2023). This approach is scalable and inexpensive, but it can under-measure semantic adequacy and holistic answer quality in open-ended generations, and it often fails to directly capture information-grounding or multi-claim factual consistency.

LLM-as-a-Judge: An increasingly prevalent paradigm in which a strong LLM (e.g., GPT-4) is prompted to score, critique, or rank system outputs against explicit criteria such as faithfulness and relevance. It can serve as a scalable proxy for human judgments, particularly for nuanced, rubric-based criteria that are hard to encode as simple metrics. Nevertheless, it introduces additional inference cost and may inherit biases from the judge model (e.g., reliance on its parametric knowledge, sensitivity to prompt phrasing, or limited objectivity). Frameworks such as ARES (Saad-Falcon et al. 2024) aim to systematize and standardize this evaluation workflow.

End-User/Task-Oriented Evaluation: The most ecologically valid form of evaluation, measuring performance within a real workflow (e.g., reduced time-to-completion, improved decision quality, or higher user satisfaction) (Vaithilingam et al. 2022). While highly meaningful for deployment, it is context-dependent, costly to run, and difficult to generalize across domains or tasks.

7.4 Benchmark Limitations and Critical Gaps

Despite rapid progress in RAG evaluation, several structural limitations and open gaps remain that hinder reliable system comparison and principled iteration (Yu et al. 2024).

Over-Reliance on Static, Wikipedia-Centric Knowledge. A large fraction of widely-used QA/RAG benchmarks are ultimately grounded in snapshots of Wikipedia (e.g., Natural Questions (Kwiatkowski et al. 2019), HotpotQA (Yang et al. 2018)), where questions are annotated against specific Wikipedia pages or paragraphs. While

these datasets are valuable for controlled benchmarking, they weakly reflect primary production settings where RAG is deployed over dynamic, proprietary, or domain-specific knowledge bases (e.g., continuously updated technical documentation, internal corporate wikis, ticketing systems, or policy repositories). The domain-focused benchmarks and corpora such as TechQA (Castelli et al. 2020), MIRAGE for medical RAG (Xiong et al. 2024), and RAGBench with industry corpora such as user manuals (Friel et al. 2024) broaden coverage beyond Wikipedia, but overall benchmark diversity is still limited and often fails to capture the full variability of enterprise data and update dynamics.

Lack of Fine-Grained, Diagnostic Benchmarks. Many benchmarks collapse performance into a single aggregate score, which is insufficient for debugging modular RAG pipelines. A key missing capability is diagnostic evaluation that supports error attribution: distinguishing whether a failure is caused by retrieval (e.g., missing the crucial passage), comprehension (e.g., misreading a number/date), or reasoning (e.g., failing to integrate multiple retrieved information snippets). Recent efforts move toward fine-grained diagnosis—for example, RAGChecker introduces module-aware diagnostic metrics (Ru et al. 2024). However, such diagnostic supervision typically requires more granular annotation (e.g., retrieved information mapping, claim-level grounding), which is expensive and difficult to scale.

Under-exploration of Efficiency and Cost. Evaluation still overwhelmingly prioritizes answer quality while under-reporting operational metrics that determine deployability: end-to-end latency (including p95/p99), time-to-first-token (TTFT), throughput, memory footprint, and dollar cost per query. Systems studies show that RAG introduces non-trivial latency–throughput–memory trade-offs and can substantially increase TTFT and storage demands, making quality-only comparisons incomplete (Shen et al. 2024; Jiang et al. 2025). Consequently, a "better" RAG method under offline quality metrics may be impractical under real serving constraints.

Insufficient Focus on Multi-Modal and Complex Retrieval. As RAG expands beyond plain text to PDFs with tables/figures, images, and structured backends (e.g., SQL databases), evaluation frameworks lag behind and lack consistent, standardized protocols across modalities. Emerging benchmarks highlight this gap, such as MRAG-Bench for vision-centric retrieval-augmented multimodal models (Hu et al. 2025a) and the multimodal RAG Benchmark built from PDF pages (Peng et al. 2025). For table-heavy and heterogeneous settings, recent benchmarks and tasks (e.g., TARGET for table retrieval (Ji et al. 2025), and TableRAG for retrieval plus SQL-style reasoning over heterogeneous documents (Yu et al. 2025)) represent important progress, but evaluation remains fragmented and modality-specific rather than unified.

Temporal Dynamics and Knowledge Freshness. Few benchmarks explicitly test temporal reasoning and the ability to prioritize up-to-date information, despite this being crucial for real deployments (e.g., news, finance, policy compliance). Time-sensitive QA benchmarks and protocols such as the Time-Sensitive QA dataset (Chen et al. 2021), FreshQA (Vu et al. 2024), and contamination-resistant UnSeenTimeQA (Uddin et al. 2025)) show that models often struggle when facts evolve or questions require temporal debunking. On the retrieval side, recent benchmarks such as TEMPO explicitly combine temporal reasoning with reasoning-intensive retrieval and introduce

temporal-specific retrieval metrics (Abdallah et al. 2026), but such evaluations are still not mainstream in end-to-end RAG benchmarking.

Overall, RAG evaluation is shifting from reusing classical information retrieval and question answering metrics toward multi-dimensional, module-aware frameworks that emphasize grounding/faithfulness, robustness, and diagnostic usefulness (Yu et al. 2024; Ru et al. 2024; Friel et al. 2024). Future evaluation suites should further close the gaps in (i) diagnostic error attribution, (ii) efficiency–cost reporting under realistic serving constraints, (iii) multimodal and structured retrieval, and (iv) temporal freshness. A robust evaluation suite is not only an assessment instrument but also a development roadmap for building RAG systems that are reliable, efficient, and trustworthy.

8 RAG Training and KB Update

This section introduces RAG training, which can be categorized into two main classes: **RAG without knowledge base update** and **RAG with knowledge base update**. The former refers to the case where only trainable parameters in each module of RAG would be updated, and the knowledge in the KB would remain the same during the training stage. The latter refers to the case where the knowledge in the knowledge base would be updated, then each module's parameters in RAG would be updated in a similar way as the former case.

8.1 RAG without Knowledge Base Update

The goal of training RAG without a knowledge base update is to update the knowledge stored in the short-term memory of generators based on the existing knowledge base. As shown in Figure 3 (a)-(c), there are three training cases, i.e., training the retriever, training the generator, and jointly training the retriever and generator.

8.1.1 Training Retriever.

Considering the case of no knowledge base update, training the retriever generally refers to training the retriever encoder and rebuilding the indexing. Since sparse encodings rely on statistical methods without parameters, training the encoder pertains only to dense encoding methods. Different training methods may have different goals, such as improving the semantic representations, accelerating the encoding process, or learning the domain-specific representations. The first two goals are often achieved by replacing the original encoder with a more powerful or tiny encoder, such as DistilBERT (Sanh et al. 2019), or TinyBERT (Jiao et al. 2020). The last requires training the original encoder on the domain-specific corpus. REPLUG (Shi et al. 2024) updated the retriever by minimizing KL divergence between retrieval and LM scores, and asynchronously refreshing the knowledge base index. After training the retriever encoder, the vector database's embeddings that serve as keys will also change. Thus, all indexes should be rebuilt with new embeddings. Besides, if the encoder remains unchanged, the indexing can be updated using new ANN searching algorithms or re-tuning the hyperparameters. After the retriever is trained, it can be directly incorporated into the RAG without updating the generator.

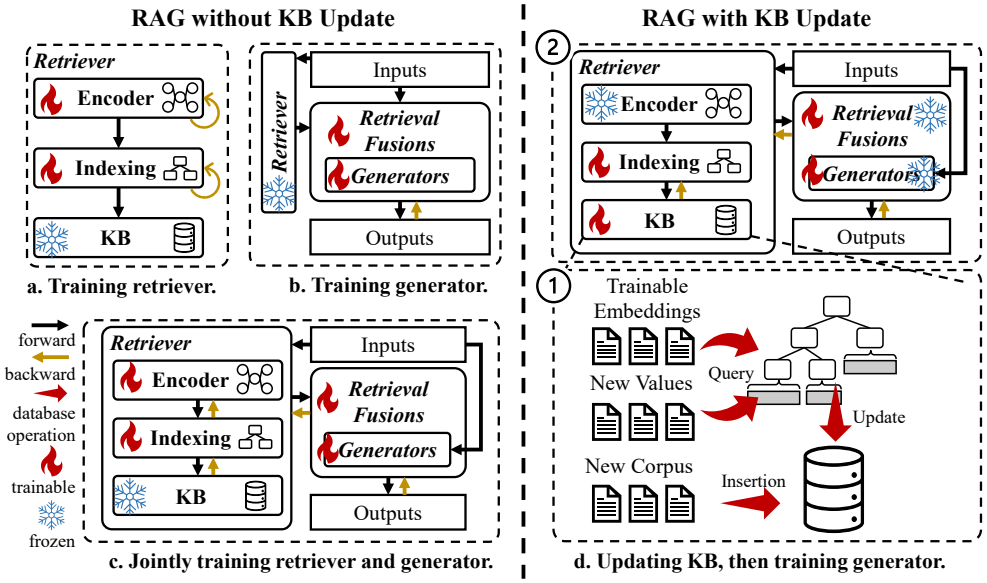

Fig. 3 Different RAG training strategies with/without knowledge base update.

8.1.2 Training Generator.

Training the generator involves updating its parameters or those in the retrieval fusion modules. Since the generator is generally an LLM, training the LLM is a resource- and time-consuming process. Fortunately, several parameter-efficient fine-tuning techniques, such as LoRA (Hu et al. 2022), are proposed to address the fine-tuning problem of LLMs. Although the parameters in the retrieval fusion modules are less than those in the generator, only fine-tuning those parameters may encounter some training problems, such as low convergence and overfitting. Jointly tuning the parameters in the generator and the retrieval fusion modules is a better way to train the generator and the retrieval fusion modules if there are sufficient and powerful resources.

8.1.3 Jointly Training the Retriever and Generator.

Apart from independently training the retriever and the generator, jointly training the retriever and the generator can be another good choice for better performance on downstream tasks. The primary challenges involve computational complexity from large-scale retrieval and marginalization over latent documents, alongside training instability due to potential retrieval collapse and interdependent retriever-generator feedback loops. Additionally, sparse gradients from discrete retrieval steps and approximation errors from fixed top-k document selection limit effective end-to-end optimization. Typically, complex indexes, such as FAISS (Douze et al. 2024), are not a suitable choice during the fine-tuning stage.

Existing works generally leverage the complex indexes to pre-select a small subset of nearest neighbors as candidates, then choose the final top-k nearest neighbors

by performing the matrix-multiplication operations. REALM (Guu et al. 2020) and RAG (Lewis et al. 2020) both unify retriever and language modeling by treating retrieved documents as latent variables and optimizing end-to-end via gradient descent. They both used Maximum Inner Product Search (MIPS). RAG (Lewis et al. 2020) only updated the encoder in the retriever, but REALM (Guu et al. 2020) also asynchronous refreshed the indexing. Atlas (Izacard et al. 2023) jointly pre-trained the retriever and generator, using different loss functions (such as attention distillation and perplexity distillation) to optimize the retriever. Besides, it also asynchronous refreshed the indexing. Experimental results in these works demonstrate that joint training is an end-to-end optimization that can lead to better coordination between the retriever and the generator and improve the contextual understanding of the generator.

8.2 RAG with Knowledge Base Update

As shown in Figure 3 (d), the scenario involves two stages: updating the knowledge base, then training the retriever and the generator. There are three cases for updating the knowledge base, i.e., updating with trainable embeddings, updating with new values, and updating with a new corpus. In the first case, values generally are trainable embeddings and are simultaneously/asynchronously updated with parameters in the RAG (Chen et al. 2022b). The last two cases usually refer to updating the knowledge base with up-to-date information. Taking the question-answer corpus as an example, updating with new values refers to updating the answer to existing questions, while updating with a new corpus refers to adding new question-answer pairs. To update the value of existing keys requires first querying the existing key-value pairs and then performing in-place updates. For a new corpus, the knowledge base first needs to perform insertion operations, then rebuild or update the indexes as new keys are added. After updating the knowledge base, training the retriever and the generator is similar to RAG without knowledge base updates. However, this training step is not always necessary, thanks to the in-context learning capability of LLMs.

9 Applications

9.1 RAG in Industry: An Agent-Centric View

LLM-based autonomous agents are intelligent software systems that leverage the power of LLMs to perform tasks without continuous human intervention (Li et al. 2024a; Xi et al. 2025; Wang et al. 2024b). These agents use LLMs as a brain or controller (Huang et al. 2024), and extend their abilities through multimodal perception (Xie et al. 2024), tool utilization (Schick et al. 2023), and external memory (Packer et al. 2023). In industrial deployments, RAG is widely adopted as the grounding layer for agentic systems (copilots/assistants), because it enables agents to (i) consult proprietary knowledge with provenance and (ii) update knowledge without retraining the model (Lewis et al. 2020; Microsoft 2026b). Moreover, recent enterprise RAG practices increasingly emphasize governed retrieval rather than unrestricted open-web browsing (Microsoft 2025c).

Using RAG to Retrieve from External Memory. LLM-based agents can utilize RAG to access and retrieve information from their own external memory (Hatalis et al. 2023; Zhang et al. 2025; Mei et al. 2024). This external memory serves as a knowledge base that the agent can draw upon to enhance its understanding and decision-making. When faced with a query or a task, the agent retrieves relevant memory items and integrates them into the generation process of the LLM. This allows the agent to produce responses or solutions informed by a wider range of knowledge, leading to more accurate and contextually relevant outcomes.

Using Tools and RAG for Up-to-Date Information. In addition to retrieving information from its own memory, an agent can use tools to acquire fresh information, which is useful for tasks requiring up-to-date knowledge (e.g., market and policy updates) (Schick et al. 2023). In enterprise settings, such retrieval is often implemented via governed interfaces (e.g., enterprise search/connectors) to preserve permissions and compliance. For example, the Microsoft 365 Copilot Retrieval API powers applications that access content from SharePoint, OneDrive, and Copilot connectors while maintaining secure access controls. For complex queries in copilot apps, agentic retrieval further decomposes a question into multiple focused subqueries and aggregates results for downstream answer synthesis (Microsoft 2025c). Overall, RAG augments agent capabilities by enabling access to broader and fresher information from memory and external sources, improving decision-making and reliability.

9.2 Frameworks and Libraries for RAG Application Development

The practical implementation of RAG has been greatly accelerated by the emergence of specialized software libraries and orchestration frameworks. While often colloquially referred to as platforms, these tools are more accurately described as development frameworks that provide the necessary primitives for building, managing, and optimizing RAG pipelines.

The foundational libraries in this space include LangChain (LangChain 2023) and LlamaIndex (Liu 2022), which pioneered the modular composition of retrieval, augmentation, and generation components. Since their introduction, the ecosystem has diversified significantly to accommodate different architectural preferences and language stacks. For instance, Haystack (deepset 2026) offers a more production-oriented approach by treating RAG pipelines as first-class primitives. In contrast, frameworks like Semantic Kernel (Microsoft 2025a) and AutoGen (Microsoft 2024) integrate retrieval capabilities directly into agentic workflows, with AutoGen featuring specialized agents such as RetrieveChat for complex, multi-step RAG interactions.

For developers seeking higher-level control, DSPy (DSPy 2026) abstracts RAG pipelines into programmable modules, enabling systematic optimization of prompts and retrieval strategies. Similarly, Prompt flow (Microsoft 2025b) facilitates the development lifecycle within cloud environments like Azure ML. The ecosystem has also expanded beyond Python, with Spring AI (Spring 2026) providing modular RAG and ETL support for Java-based enterprise systems, and the Vercel AI SDK (Vercel 2026) offering tailored solutions for TypeScript-centric, front-end applications. Collectively,

these tools represent a shift from bespoke RAG implementations to standardized, reusable, and often language-agnostic development paradigms.

9.3 Deployment Considerations for Production RAG

Although the RAG pattern is conceptually simple, production systems face multi-dimensional constraints, including retrieval quality, scalability, token/context budgets, latency, security/governance, and evaluation/observability (Microsoft 2026b). In this section, we will discuss the deployment considerations for production RAG from four following aspects:

Scalability Challenges and Retrieval Infrastructure. Large-scale deployments require efficient vector indexing and ANN search. HNSW is a widely used ANN structure with favorable scaling properties (Malkov and Yashunin 2020), and GPU-accelerated similarity search has been studied for billion-scale retrieval workloads (Johnson et al. 2021). Industry vector search systems further provide operational features (endpoints, governance, budget policies, etc.) (Databricks 2025a).

Retrieval Quality Optimization and System Knobs. Production quality improvements are typically staged and evaluation-driven. Databricks' retrieval quality guide recommends establishing an evaluation framework first, then applying hybrid search, metadata filtering, and reranking as progressively more expensive steps (Databricks 2025b). Similarly, Azure distinguishes classic RAG and agentic retrieval for complex queries, reflecting a practical trade-off between quality and latency (Microsoft 2026b,a).

Cost-Performance Trade-Offs. Key cost drivers include ingestion/embedding, vector search, reranking, and LLM token usage. Vertex AI RAG Engine explicitly itemizes billing across ingestion/parsing, embeddings, vector search (managed Spanner), and reranking, enabling cost attribution and budgeting (Google Cloud 2026b). Token budgets further motivate careful top-k selection and context compression (Microsoft 2026b).

Production System Requirements Enterprise deployments require Access Control List (ACL) aware retrieval and compliance controls (Microsoft 2025c). Provenance (citations) improves trust and auditability. Amazon Bedrock Knowledge Bases returns citations to specific retrieved chunks in Retrieve-and-Generate responses (Amazon Web Services 2026). Evaluation and monitoring are essential to prevent silent regressions; OpenAI provides an Evals API to run evaluation jobs programmatically (OpenAI 2026), and LangSmith provides observability for tracing and evaluation of LLM applications (LangChain 2026). Managed RAG platforms may also provide security controls such as VPC-SC and CMEK support (Google Cloud 2026a).

10 Discussion and Future Directions

Despite the success of the RAG for natural language processing, some challenges should be considered. This paper highlights these challenges to inspire future research and provides possible future research directions in RAG for NLP.

10.1 Security and Privacy Considerations for RAG

RAG improves factuality by grounding generation in external data sources, but it also introduces new security and privacy risks because the retrieval database (e.g., document store or vector database) becomes an additional trust boundary and attack surface. Recent studies show that manipulating or contaminating the knowledge base can directly steer downstream generation, indicating that RAG security cannot be reduced to the base LLM's safety alone (Zou et al. 2025). In practice, design choices discussed earlier, such as retrieving larger top-k sets, adopting early-fusion concatenation, or using query rewriting in query-based fusion, may amplify exposure by increasing the amount of external content injected into the model context, thereby enlarging the attack surface.

Privacy Risks of External Knowledge Bases. RAG systems typically store either raw documents, embeddings, or both. A common assumption is that embeddings are "safe" representations; however, research (Li et al. 2023a) demonstrates that sentence embeddings can leak substantial information and can be inverted to recover original sentences, which challenges the privacy-by-embedding assumption. Furthermore, OWASP (owa 2025b) highlights that multi-tenant vector databases may suffer from cross-context leakage if access control and dataset partitioning are inadequate, allowing unauthorized retrieval across users or applications. To mitigate such risks, permission-aware retrieval (e.g., RBAC-filtered retrieval (Martinelli and Ponti 2025)) has been proposed to enforce least-privilege access at retrieval time, preventing sensitive passages from entering the prompt in the first place. Complementary measures can be using strong tenant isolation, auditing of retrieval logs, and designing privacy-preserving embedding defense.

Information Leakage Through Retrieval and Generation. Even when the knowledge base is not directly compromised, RAG may leak private or proprietary information through the retrieval-then-generation pipeline. Attackers can craft adaptive queries to induce the system to reveal sensitive sentences that originate from the knowledge base, and recent work (Chen et al. 2025b) proposes black-box frameworks for fine-grained privacy extraction that explicitly identify and extract knowledge-base-derived sensitive sentences from mixed RAG outputs. Other studies (Cohen et al. 2024) further show that, under jailbreaking capabilities, attacks can be escalated toward large-scale document extraction from RAG knowledge bases, with leakage severity affected by context length and embedding configurations. These results imply that improving retrieval recall (e.g., larger top-k) or aggressive fusion can trade off against confidentiality. Practical mitigations include retrieval-time authorization, sensitive-content filtering/redaction on retrieved chunks, and limiting verbatim reproduction of retrieved passages (e.g., summary-first with controlled quotation). Additionally, using synthetic data to replace sensitive corpora has been explored as a privacy-oriented alternative for RAG augmentation (Zeng et al. 2025).

Adversarial Attacks on RAG Systems and Defenses. Attackers can target the knowledge base, the retriever, or the interaction between retrieval and generation via knowledge poisoning or prompt injection. Knowledge poisoning injects malicious texts into the knowledge base to manipulate outputs for targeted queries (Zou et al. 2025; Chang et al. 2025). Prompt injection remains another central threat: indirect

injection can be embedded in retrieved documents, blurring the line between data and instructions (owa 2025a). Defenses can be organized into three layers: (i) ingestion-time provenance and access control, (ii) retrieval-time anomaly detection, and (iii) generation-time verification and correction. Representative approaches include Self-RAG (Asai et al. 2024) that retrieves on demand with self-critique, RARR (Gao et al. 2023a) that leverages post-hoc revision for attribution, and SelfCheckGPT (Manakul et al. 2023) that introduces sampling-based consistency checks. Fine-grained citations further improve auditability by grounding claims in retrieved information (Xia et al. 2025).

Security Evaluation for RAG. Given these risks, RAG evaluation should extend beyond answer accuracy and retrieval metrics to include security/privacy robustness (e.g., leakage rate, prompt-injection success rate, poisoning success rate). Benchmarks such as SafeRAG (Liang et al. 2025) facilitate systematic security evaluation of RAG pipelines. For defenses, recent research explores instruction detection and removal pipelines for indirect prompt injection, but results also suggest that robustness is far from solved, motivating future work on principled trust modeling of retrieval sources, provenance-aware ingestion, and secure fusion strategies that minimize exposure while preserving utility (Chen et al. 2025a).

10.2 Retrieval Quality

High-quality retrieval is fundamental to RAG systems. Imperfect retrieval, whether due to missing relevant passages, returning contradictory information, or introducing noise, can directly degrade generation quality and even induce hallucinations (Park and Lee 2024). Ensuring high retrieval quality requires careful consideration of both single-round retrieval factors (e.g., key representation, embedding models) and the key challenges in multi-hop or iterative retrieval settings (e.g., semantic drift).

Improving Single-Round Retrieval Quality. For each retrieval, four key factors collectively determine the quality of retrieved results. The first is the key choice in retriever, which should be aligned with the downstream task. For example, for QA tasks, adopting the question itself as the key is an intuitive yet effective choice. The second factor is the embedding model, which converts text into vector representations. General-purpose models like BERT (Devlin et al. 2019) or RoBERTa (Liu et al. 2019) may suffice for broad domains, but adapting the encoder to the target domain can substantially improve semantic matching. Third, the similarity metric used to compare query and key embeddings plays a critical role. Beyond standard cosine or Euclidean distance (Gunawardana and Shani 2009), task-specific metrics such as optimal transport distance (Cui et al. 2023) have been shown to better capture nuanced relevance in specific domain scenarios. Finally, the choice of ANN search algorithm (e.g., HNSW (Malkov and Yashunin 2020), IVF-PQ (Jégou et al. 2011)) introduces a trade-off between search speed and accuracy; tuning its parameters directly affects both the precision of retrieved results and system latency.

Semantic Drift in Multi-hop or Iterative Retrieval. Beyond single-round retrieval, many RAG systems employ multi-hop or iterative retrieval strategies to gather retrieved information incrementally. For example, decomposing a complex question into sub-queries, or reformulating queries based on initial retrieval results.

While such strategies can improve coverage, they introduce the risk of *semantic drift* issues (Zhu et al. 2025), that is as the system retrieves multiple rounds of information, the retrieved content may gradually deviate from the original information need. This occurs because each retrieval step relies on the output of the previous step, and small errors or off-topic tangents can compound over iterations. Semantic drift might easily occur when any of the single-round retrieval factors are suboptimal. For example, a weakly aligned embedding model may return some irrelevant knowledge that steer subsequent queries off course. Consequently, mitigating semantic drift requires not only optimizing each individual retrieval step but also designing mechanisms to monitor and correct for drift, such as query validation (Roy et al. 2024), relevance feedback loops (Mao et al. 2024; Kim and Lee 2024), or early stopping when retrieved information becomes consistently irrelevant (Jiang et al. 2023b; Su et al. 2024b; Park et al. 2025; Cheng et al. 2024).

10.3 RAG Efficiency

RAG efficiency is crucial for downstream NLP applications, which limits the volume of data that can be retrieved. There are two simple ways to guarantee RAG efficiency without new algorithms, i.e., reducing the volume of data or adding more powerful computing and memory resources. However, the former may impact the retrieval quality, while the latter requires more resource cost.

RAG efficiency encompasses the efficiency of the retriever and the efficiency of retrieval fusions. Retriever efficiency refers to the time cost of retrieving relevant information, which can be divided into three parts, i.e., encoding time, ANN searching time, and data fetching time of the knowledge base. It is unnecessary to jointly optimize all three components as the bottleneck would vary from different database sizes. For smaller retrieval databases, such as those with fewer than 1 million entries, the encoding phase is often the primary bottleneck, as the vector database can be all stored in the memory. Several topics, such as model quantization (Kim et al. 2021; Bai et al. 2021), distillation (Jiao et al. 2020; Ding et al. 2023), or model pruning (Ganesh et al. 2021), are used to accelerate the encoding. Besides, efficient LLM serving systems (e.g., vLLM/PagedAttention) and decoding accelerators (e.g., speculative decoding) can significantly improve throughput or reduce decoding time, helping offset RAG's longer prompts. (Kwon et al. 2023)

In contrast, for larger databases, the time cost of searching in the index and fetching data from the knowledge base becomes the major bottleneck, as the searching is over a considerable amount of data, and the fetching involves I/O overheads. In this case, efficient ANN searching algorithms (Johnson et al. 2021; Douze et al. 2024; Guo et al. 2020) and system-level optimizations (Jin et al. 2024a; Jiang et al. 2024b) are the main focus.

Retrieval fusion efficiency, which aims to enhance the inference efficiency when integrating retrievals, is worth optimizing for improving the RAG efficiency. For example, the computational overhead of query-based fusion is often non-negligible due to the long sequence length. Some works, such as Fid-light (Hofstätter et al. 2023) and ReFusion (Wu et al. 2024), mainly target reducing the computations while integrating the retrieved information.

10.4 RAG Reliability

Reliability is another practical challenge. RAG/agentic variants may exhibit repetitive generation or "looping" behaviors that stall responses. Degenerative repetition has been analyzed in neural text generation and is closely related to decoding dynamics (Holtzman et al. 2020). Production studies further report that repetition can cause severe system stalling and summarize effective countermeasures such as early-stopped beam search, decoding penalties, and training-time alignment (Wang et al. 2025b). Decoding strategies such as contrastive search provide additional tools to reduce repetition and improve coherence (Su and Collier 2023). In practice, robust termination criteria (max steps/budgets, loop detectors, stop sequences), coupled with monitoring and fallback policies (e.g., reduced context, simpler answer mode), are essential for production-grade reliability.

10.5 RAG vs. Long-Context LLMs

The emergence of long-context LLMs, capable of processing millions of tokens in a single forward pass, has raised a fundamental question: does RAG still matter when the model can directly inject the context of the entire knowledge base? This section examines this question from two angles. First, we compare RAG and long-context LLMs as competing paradigms, analyzing their relative strengths and weaknesses. Second, we explore how RAG can complement long-context LLMs to achieve further improvements, rather than being rendered obsolete.

RAG vs. Long-Context LLMs as Competing Paradigms Recent studies have compared the performance of RAG systems and long-context LLMs across various tasks. Hui et al. (Hui et al. 2024) demonstrate that on free-form or knowledge-based tasks (e.g., paper-based and wiki-based QA), both paradigms perform similarly. However, on tasks requiring numerical reasoning (e.g., financial QA), long-context LLMs underperform compared to RAG, as the verbose content in long contexts can obscure key facts and hinder precise reasoning. Moreover, for smaller LLMs that struggle to process large volumes of data effectively, RAG tends to outperform long-context mechanisms. Beyond performance, long-context LLMs usually incur significantly higher computational overhead as input length grows, while RAG processes only selected relevant passages, offering a more economical solution. These findings suggest that while long-context capability is valuable, it does not fully replace the need for retrieval, especially when precision, reasoning over scattered facts, resource efficiency, or costs are priorities.

Enhancing Long-Context LLMs with RAG. Rather than regarding them as mutually exclusive, RAG can be used to further improve long-context LLMs. Instead of feeding the entire knowledge base into the context window, a retriever selects only the most relevant information which acts as a context compressor, reducing computational cost and mitigating the "lost in the middle" issue (Liu et al. 2024). To fully realize synergies, several improvement strategies can be adopted when applying RAG to long-context LLMs. For example, lightweight prompt engineering (Lewis et al. 2020; Guo et al. 2023) or calibration methods (Jiang et al. 2023b; Asai et al. 2024) can offer improved results with minimal overhead; iterative RAG (Shao et al. 2023) or query

refinement (Yan et al. 2024) can achieve better performance and robustness. These strategies can also be potentially combined, e.g., refined queries with iterative loops. In essence, long-context LLMs and RAG are not competitors but complementary tools. Long context provides breadth while retrieval provides precision and verifiability.

10.6 RAG Training

As introduced in Section 8, RAG training includes two branches of works, RAG with-/without knowledge base update. For RAG without a knowledge base update, the main challenge is how to jointly optimize all parameters in RAG. This may involve new loss functions with multiple objectives, new optimizations for efficient tuning parameters in the retriever and generator, or other training strategies.

For RAG with knowledge base update, one challenge is how to align the retrieval representations with the generator's representations. Although the time cost of the update operation in knowledge base cannot be ignored, some works (Chen et al. 2022b) reduce the update frequency by asynchronously updating, thus achieving the alignment of knowledge representation and model's representation. Another challenge is when to retrain/fine-tune the generator in RAG when a new corpus is added. Due to the in-context learning capability of existing LLM-based generators and high training overhead, retraining/fine-tuning the generator or directly inferring the generator becomes a challenging choice for different scenarios. Recently, some efficient training strategies (Hu et al. 2022; Dettmers et al. 2023) have been proposed to accelerate the fine-tuning process, which can be taken into consideration.

10.7 Cross-Modality Retrieval

Retrieving cross-modality information in NLP tasks can greatly enhance the quality and richness of the representations, leading to improved performance. First, cross-modality information, such as combining text with images, videos, or audio, provides a richer context to the content (Hu 2023). For instance, when language is ambiguous, accompanying images can clarify meanings difficult to convey through text alone. Second, different modalities can contribute various types of information that are not accessible from a single source. For example, visual data can provide spatial, color, and action cues, while textual data can offer detailed descriptions, emotions, or abstract concepts. Combining these can lead to a more comprehensive understanding of the data. Moreover, Models trained on multi-modal data typically exhibit increased robustness and generalizability (Wu and Goodman 2018). These models are adept at associating information across diverse inputs, mitigating overfitting to the peculiarities of a single modality. This attribute is particularly valuable in real-world applications of NLP, such as in autonomous vehicles, where systems must interpret textual information from signs or dialogues and sensory data from the surrounding environment to make informed decisions. Furthermore, multi-modal data can resolve ambiguities that cannot be resolved within a single modality. For example, the phrase "bank" can refer to either a financial institution or the side of a river, and visual context can help disambiguate this. Last, human communication is inherently multi-modal, incorporating elements such as gestures, facial expressions, and tone of voice. Systems capable

of processing multiple modes of communication can interact with humans in a manner that is both more natural and intuitive. In conclusion, integrating cross-modality information in RAG for NLP tasks not only enhances the richness and quality of data representations but also significantly improves the systems' comprehension, interaction capabilities, and adaptability to diverse applications.

10.8 GraphRAG and Graph-based Retrieval

Recent open-source and research advances have pushed RAG beyond flat passage retrieval toward graph-based retrieval, where documents are transformed into structured entity–relation representations and retrieval operates over graph neighborhoods or communities. A prominent example is GraphRAG, which uses an LLM to build a graph index in two stages: (i) constructing an entity knowledge graph from source documents, and (ii) pre-generating community summaries for groups of closely related entities. At query time, community summaries are used to produce partial answers that are then aggregated into a final response (Edge et al. 2024). Microsoft has also released an open-source GraphRAG implementation, enabling practitioners to reproduce and extend graph-based pipelines, while noting that graph indexing can be operationally expensive [2]. At the same time, recent analyses emphasize that graph-based RAG does not uniformly dominate "vanilla" RAG. The benefits depend on the task structure (e.g., hierarchical or multi-hop reasoning) and may entail higher latency and graph construction overhead, motivating careful scenario-based evaluation and benchmarking (Xiang et al. 2025). den

11 Conclusion

In this survey, we presented a systematic review of RAG for NLP. We introduced its core components, including retriever, generator, and retrieval fusions, and provided a novel taxonomy of fusion methods with structured comparisons across accessibility, efficiency, and use cases. We further examined RAG applications across diverse NLP tasks and discussed evaluation methodologies and benchmark limitations. Training paradigms with and without knowledge base updates were also analyzed. Finally, we explored industrial deployment considerations and identified emerging challenges and future directions, such as security or graph-based retrieval. We hope this survey serves as a practical guide for researchers and practitioners advancing RAG systems.

12 Statements and Declarations

12.1 Authors' Contribution

S.W. and Y.X., wrote and significantly revised the main manuscript. Y.C., H.W., C.C., and Y.Y. drafted several sections. S.W., Y.X., Y.C., H.W., C.C., Y.Y, L.H, X.L, T.K, N.G, and C.X participated in the main investigation and the design of the paper's structure. All authors participated in designing and refining the methodology. All authors reviewed the manuscript.

[2] https://github.com/microsoft/graphrag

12.2 Conflict of Interest

The authors declare no Conflict of interest.

References

(2025a) Llm01:2025 prompt injection - owasp gen ai security project. URL https://genai.owasp.org/llmrisk/llm01-prompt-injection/

(2025b) Llm08:2025 vector and embedding weaknesses - owasp gen ai security project. URL https://genai.owasp.org/llmrisk/llm082025-vector-and-embedding-weaknesses/

Abdallah A, Ali M, Abdul-Mageed M, et al (2026) TEMPO: A realistic multi-domain benchmark for temporal reasoning-intensive retrieval. CoRR abs/2601.09523

Abnar S, Dehghani M, Neyshabur B, et al (2022) Exploring the limits of large scale pre-training. In: Proceedings of The Tenth International Conference on Learning Representations (ICLR)

Ainslie J, Lee-Thorp J, de Jong M, et al (2023) GQA: training generalized multi-query transformer models from multi-head checkpoints. In: Bouamor H, Pino J, Bali K (eds) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 4895–4901

AlKhamissi B, Li M, Celikyilmaz A, et al (2022) A review on language models as knowledge bases. CoRR abs/2204.06031

Amazon Web Services (2026) Retrieving information from data sources using amazon bedrock knowledge bases. https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-retrieval.html, accessed: 2026-03-04

Anil R, Borgeaud S, Wu Y, et al (2023) Gemini: A family of highly capable multimodal models. CoRR abs/2312.11805

Arefeen MA, Debnath B, Chakradhar S (2024) Leancontext: Cost-efficient domain-specific question answering using llms. Natural Language Processing Journal 7:100065. https://doi.org/10.1016/J.NLP.2024.100065

Asai A, Wu Z, Wang Y, et al (2024) Self-rag: Learning to retrieve, generate, and critique through self-reflection. In: Proceedings of The Twelfth International Conference on Learning Representations (ICLR)

Baek J, Aji AF, Saffari A (2023) Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. CoRR abs/2306.04136

Bai H, Zhang W, Hou L, et al (2021) Binarybert: Pushing the limit of BERT quantization. In: Proceedings of the 59th Annual Meeting of the Association for

Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 4334–4348

Bertsch A, Alon U, Neubig G, et al (2023) Unlimiformer: Long-range transformers with unlimited length input. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Borgeaud S, Mensch A, Hoffmann J, et al (2022) Improving language models by retrieving from trillions of tokens. In: Proceedings of the 39th International Conference on Machine Learning (ICML), pp 2206–2240

Brown TB, Mann B, Ryder N, et al (2020) Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33 (NeurIPS)

Cai D, Wang Y, Li H, et al (2021) Neural machine translation with monolingual translation memory. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 7307–7318

Castelli V, Chakravarti R, Dana S, et al (2020) The techqa dataset. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1269–1278

Chalkidis I, Fergadiotis M, Malakasiotis P, et al (2020) LEGAL-BERT: the muppets straight out of law school. In: Findings of the Association for Computational Linguistics: EMNLP, pp 2898–2904

Chang Z, Li M, Jia X, et al (2025) One shot dominance: Knowledge poisoning attack on retrieval-augmented generation systems. In: Findings of the Association for Computational Linguistics: EMNLP, pp 18811–18825

Chen J, Chen Q, Li D, et al (2022a) Sedr: Segment representation learning for long documents dense retrieval. CoRR abs/2211.10841

Chen J, Lin H, Han X, et al (2024) Benchmarking large language models in retrieval-augmented generation. In: Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI), pp 17754–17762

Chen W, Wang X, Wang WY (2021) A dataset for answering time-sensitive questions. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)

Chen X, Li L, Zhang N, et al (2022b) Decoupling knowledge from memorization: Retrieval-augmented prompt learning. In: Advances in Neural Information Processing Systems 35 (NeurIPS)

Chen Y, Li H, Sui Y, et al (2025a) Can indirect prompt injection attacks be detected and removed? In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp 18189–18206

Chen Y, Wang Y, Zhang H, et al (2025b) Fine-grained privacy extraction from retrieval-augmented generation systems via knowledge asymmetry exploitation. CoRR abs/2507.23229

Chen Y, Xiong Y, Wu S, et al (2025c) Beyond semantic similarity: Reducing unnecessary API calls via behavior-aligned retriever. CoRR abs/2508.14323. https://doi.org/10.48550/ARXIV.2508.14323

Chen Y, Xiong Y, Wu S, et al (2026) Refilter: Improving robustness of retrieval-augmented generation via gated filter. arXiv preprint arXiv:260212709

Cheng Q, Li X, Li S, et al (2024) Unified active retrieval for retrieval augmented generation. In: Findings of the Association for Computational Linguistics: EMNLP, pp 17153–17166, https://doi.org/10.18653/V1/2024.FINDINGS-EMNLP.999

Cheng X, Lin Y, Chen X, et al (2023a) Decouple knowledge from paramters for plug-and-play language modeling. In: Findings of the Association for Computational Linguistics: ACL, pp 14288–14308

Cheng X, Luo D, Chen X, et al (2023b) Lift yourself up: Retrieval-augmented text generation with self-memory. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Chevalier A, Wettig A, Ajith A, et al (2023) Adapting language models to compress contexts. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 3829–3846, https://doi.org/10.18653/V1/2023.EMNLP-MAIN.232

Chuang Y, Fang W, Li S, et al (2023) Expand, rerank, and retrieve: Query reranking for open-domain question answering. In: Findings of the Association for Computational Linguistics: ACL, pp 12131–12147

Clark K, Luong M, Le QV, et al (2020) ELECTRA: pre-training text encoders as discriminators rather than generators. In: Proceedings of the 8th International Conference on Learning Representations (ICLR). OpenReview.net

Cohen S, Bitton R, Nassi B (2024) Unleashing worms and extracting data: Escalating the outcome of attacks against rag-based inference in scale and severity using jailbreaking. CoRR abs/2409.08045

Colombo P, Pires TP, Boudiaf M, et al (2024) Saullm-7b: A pioneering large language model for law. CoRR abs/2403.03883

Cui Y, Liu Z, Chen Y, et al (2023) Retrieval-augmented multiple instance learning. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Dai Y, Zhang Z, Liu Q, et al (2023) Simple and scalable nearest neighbor machine translation. In: Proceedings of The Eleventh International Conference on Learning Representations (ICLR)

Dale D, Voita E, Barrault L, et al (2023) Detecting and mitigating hallucinations in machine translation: Model internal workings alone do well, sentence similarity even better. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 36–50

Databricks (2025a) Mosaic ai vector search. https://docs.databricks.com/gcp/en/vector-search/vector-search, accessed: 2026-03-04

Databricks (2025b) Vector search retrieval quality guide. URL https://docs.databricks.com/aws/en/vector-search/vector-search-retrieval-quality

DeepSeek-AI (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. CoRR abs/2501.12948

deepset (2026) Haystack documentation. https://docs.haystack.deepset.ai/docs/get-started, accessed: 2026-03-04

Deguchi H, Watanabe T, Matsui Y, et al (2023) Subset retrieval nearest neighbor machine translation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 174–189

Dettmers T, Pagnoni A, Holtzman A, et al (2023) Qlora: Efficient finetuning of quantized llms. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Devlin J, Chang M, Lee K, et al (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp 4171–4186

Ding Y, Fan W, Ning L, et al (2024) A survey on RAG meets llms: Towards retrieval-augmented large language models. CoRR abs/2405.06211

Ding Z, Jiang G, Zhang S, et al (2023) SKDBERT: compressing BERT via stochastic knowledge distillation. In: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI), pp 7414–7422

Doostmohammadi E, Norlund T, Kuhlmann M, et al (2023) Surface-based retrieval reduces perplexity of retrieval-augmented language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp

521–529, https://doi.org/10.18653/V1/2023.ACL-SHORT.45

Douze M, Guzhva A, Deng C, et al (2024) The faiss library. CoRR abs/2401.08281

DSPy (2026) Tutorial: Retrieval-augmented generation (rag) – dspy. https://dspy.ai/tutorials/rag/, accessed: 2026-03-04

Dubois Y, Galambosi B, Liang P, et al (2024) Length-controlled alpacaeval: A simple way to debias automatic evaluators. CoRR abs/2404.04475

Edge D, Trinh H, Cheng N, et al (2024) From local to global: A graph RAG approach to query-focused summarization. CoRR abs/2404.16130

ES S, James J, Anke LE, et al (2024) Ragas: Automated evaluation of retrieval augmented generation. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp 150–158

explosion (2016) Spacy. misc, URL https://spacy.io/

Fabbri AR, Ng P, Wang Z, et al (2020) Template-based question generation from retrieved sentences for improved unsupervised question answering. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 4508–4513

Fabbri AR, Kryscinski W, McCann B, et al (2021) Summeval: Re-evaluating summarization evaluation. Transactions of the Association for Computational Linguistics 9:391–409

Facebook (2013) RocksDB. Software, URL https://github.com/facebook/rocksdb

Fan A, Gardent C (2022) Generating full length wikipedia biographies: The impact of gender bias on the retrieval-based generation of women biographies. CoRR abs/2204.05879

Fan A, Gardent C, Braud C, et al (2021) Augmenting transformers with knn-based composite memory for dialog. Transactions of the Association for Computational Linguistics 9:82–99

Févry T, Soares LB, FitzGerald N, et al (2020) Entities as experts: Sparse memory access with entity supervision. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 4937–4951

Friel R, Belyi M, Sanyal A (2024) Ragbench: Explainable benchmark for retrieval-augmented generation systems. CoRR abs/2407.11005

Fu Y, Chen C, Chen X, et al (2024) Optimizing the number of clusters for billion-scale quantization-based nearest neighbor search. IEEE Transactions on Knowledge and Data Engineering 36(11):6786–6800

Ganesh P, Chen Y, Lou X, et al (2021) Compressing large-scale transformer-based models: A case study on BERT. Transactions of the Association for Computational Linguistics 9:1061–1080

Gao L, Dai Z, Pasupat P, et al (2023a) RARR: researching and revising what language models say, using language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 16477–16508, https://doi.org/10.18653/V1/2023.ACL-LONG.910

Gao T, Yao X, Chen D (2021) Simcse: Simple contrastive learning of sentence embeddings. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 6894–6910

Gao Y, Xiong Y, Gao X, et al (2023b) Retrieval-augmented generation for large language models: A survey. CoRR abs/2312.10997

Glass MR, Wu X, Naik AR, et al (2023) Retrieval-based transformer for table augmentation. In: Findings of the Association for Computational Linguistics: ACL, pp 5635–5648

Gong H, Shen Y, Yu D, et al (2020) Recurrent chunking mechanisms for long-text machine reading comprehension. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 6751–6761

Google Cloud (2026a) Use data ingestion with vertex ai rag engine. https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/use-data-ingestion, accessed: 2026-03-04

Google Cloud (2026b) Vertex ai rag engine billing. https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-engine-billing, accessed: 2026-03-04

Gunawardana A, Shani G (2009) A survey of accuracy evaluation metrics of recommendation tasks. Journal of Machine Learning Research 10:2935–2962

Guo D, Tang D, Duan N, et al (2019) Coupling retrieval and meta-learning for context-dependent semantic parsing. In: Proceedings of the 57th Conference of the Association for Computational Linguistics (ACL), pp 855–866

Guo R, Sun P, Lindgren E, et al (2020) Accelerating large-scale inference with anisotropic vector quantization. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp 3887–3896

Guo R, Luan X, Xiang L, et al (2022) Manu: A cloud native vector database management system. Proceedings of the VLDB Endowment 15(12):3548–3561

Guo Z, Cheng S, Wang Y, et al (2023) Prompt-guided retrieval augmentation for non-knowledge-intensive tasks. In: Findings of the Association for Computational

Linguistics: ACL, pp 10896–10912

Guu K, Lee K, Tung Z, et al (2020) Retrieval augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp 3929–3938

Harris D, Harris S (2010) Digital design and computer architecture. Morgan Kaufmann

Harris ZS (1954) Distributional structure. Word 10(2-3):146–162

Hatalis K, Christou D, Myers J, et al (2023) Memory matters: The need to improve long-term memory in llm-agents. In: Proceedings of the AAAI Symposium Series, pp 277–280

He Q, Wang Y, Wang W (2024) Can language models act as knowledge bases at scale? CoRR abs/2402.14273

Hoffmann J, Borgeaud S, Mensch A, et al (2022) Training compute-optimal large language models. CoRR abs/2203.15556

Hofstätter S, Chen J, Raman K, et al (2023) Fid-light: Efficient and effective retrieval-augmented text generation. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). ACM, pp 1437–1447

Holtzman A, Buys J, Du L, et al (2020) The curious case of neural text degeneration. In: Proceedings of the 8th International Conference on Learning Representations (ICLR)

Hoshi Y, Miyashita D, Ng Y, et al (2023) Ralle: A framework for developing and evaluating retrieval-augmented large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 52–69

Hossain N, Ghazvininejad M, Zettlemoyer L (2020) Simple and effective retrieve-edit-rerank text generation. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 2532–2538, https://doi.org/10.18653/V1/2020.ACL-MAIN.228

Hu EJ, Shen Y, Wallis P, et al (2022) Lora: Low-rank adaptation of large language models. In: Proceedings of The Tenth International Conference on Learning Representations (ICLR)

Hu W, Gu J, Dou Z, et al (2025a) Mrag-bench: Vision-centric evaluation for retrieval-augmented multimodal models. In: Proceedings of The Thirteenth International Conference on Learning Representations (ICLR)

Hu X (2023) Multimodal named entity recognition and relation extraction with retrieval-augmented strategy. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). ACM, p 3488

Hu Y, Lu Y (2024) RAG and RAU: A survey on retrieval-augmented language model in natural language processing. CoRR abs/2404.19543

Hu Z, Murthy V, Pan Z, et al (2025b) Hedrarag: Coordinating LLM generation and database retrieval in heterogeneous RAG serving. CoRR abs/2507.09138

Huang J, Ping W, Xu P, et al (2023a) RAVEN: in-context learning with retrieval augmented encoder-decoder language models. CoRR abs/2308.07922

Huang Q, Tung AKH (2023) Lightweight-yet-efficient: Revitalizing ball-tree for point-to-hyperplane nearest neighbor search. In: Proceedings of the 39th IEEE International Conference on Data Engineering (ICDE), pp 436–449

Huang Q, Fu S, Liu X, et al (2023b) Learning retrieval augmentation for personalized dialogue generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 2523–2540

Huang X, Liu W, Chen X, et al (2024) Understanding the planning of LLM agents: A survey. CoRR abs/2402.02716

Huang Y, Huang J (2024) A survey on retrieval-augmented text generation for large language models. CoRR abs/2404.10981

Huang Y, Liu D, Zhong Z, et al (2023c) knn-adapter: Efficient domain adaptation for black-box language models. CoRR abs/2302.10879. https://doi.org/10.48550/ARXIV.2302.10879

Hui Y, Lu Y, Zhang H (2024) UDA: A benchmark suite for retrieval augmented generation in real-world document analysis. In: Advances in Neural Information Processing Systems 38 (NeurIPS)

Ishiwatari S, Yao J, Liu S, et al (2017) Chunk-based decoder for neural machine translation. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1901–1912

Izacard G, Grave E (2021) Leveraging passage retrieval with generative models for open domain question answering. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp 874–880

Izacard G, Caron M, Hosseini L, et al (2022) Unsupervised dense information retrieval with contrastive learning. IEEE Transactions on Pattern Analysis and Machine

Intelligence 2022

Izacard G, Lewis PSH, Lomeli M, et al (2023) Atlas: Few-shot learning with retrieval augmented language models. Journal of Machine Learning Research 24:251:1–251:43

Jégou H, Douze M, Schmid C (2011) Product quantization for nearest neighbor search. IEEE Transactions on Pattern Analysis and Machine Intelligence 33(1):117–128

Ji X, Glenn P, Parameswaran AG, et al (2025) TARGET: benchmarking table retrieval for generative tasks. CoRR abs/2505.11545

Ji Z, Lee N, Frieske R, et al (2023a) Survey of hallucination in natural language generation. ACM Computing Surveys 55(12):248:1–248:38

Ji Z, Liu Z, Lee N, et al (2023b) RHO: reducing hallucination in open-domain dialogues with knowledge grounding. In: Findings of the Association for Computational Linguistics: ACL, pp 4504–4522

Jiang AQ, Sablayrolles A, Mensch A, et al (2023a) Mistral 7b. CoRR abs/2310.06825

Jiang AQ, Sablayrolles A, Roux A, et al (2024a) Mixtral of experts. CoRR abs/2401.04088. https://doi.org/10.48550/ARXIV.2401.04088

Jiang H, Lu Z, Meng F, et al (2022) Towards robust k-nearest-neighbor machine translation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 5468–5477

Jiang W, Zhang S, Han B, et al (2024b) Piperag: Fast retrieval-augmented generation via algorithm-system co-design. CoRR abs/2403.05676

Jiang W, Subramanian S, Graves C, et al (2025) RAGO: systematic performance optimization for retrieval-augmented generation serving. In: Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA), pp 974–989

Jiang Z, Xu FF, Gao L, et al (2023b) Active retrieval augmented generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 7969–7992

Jiao X, Yin Y, Shang L, et al (2020) Tinybert: Distilling BERT for natural language understanding. In: Findings of the Association for Computational Linguistics: EMNLP, pp 4163–4174

Jin C, Zhang Z, Jiang X, et al (2024a) Ragcache: Efficient knowledge caching for retrieval-augmented generation. CoRR abs/2404.12457

Jin J, Wang H, Zhang H, et al (2024b) DVD: dynamic contrastive decoding for knowledge amplification in multi-document question answering. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP),

pp 4624–4637

Jin Q, Kim W, Chen Q, et al (2023) Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. Bioinformatics 39(10)

Johnson J, Douze M, Jégou H (2021) Billion-scale similarity search with gpus. IEEE Transactions on Big Data 7(3):535–547

de Jong M, Zemlyanskiy Y, FitzGerald N, et al (2022) Mention memory: incorporating textual knowledge into transformers through entity mention attention. In: Proceedings of The Tenth International Conference on Learning Representations (ICLR)

de Jong M, Zemlyanskiy Y, Ainslie J, et al (2023a) Fido: Fusion-in-decoder optimized for stronger performance and faster inference. In: Findings of the Association for Computational Linguistics: ACL, pp 11534–11547

de Jong M, Zemlyanskiy Y, FitzGerald N, et al (2023b) Pre-computed memory or on-the-fly encoding? A hybrid approach to retrieval augmentation makes the most of your compute. In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp 7329–7342

Kaplan J, McCandlish S, Henighan T, et al (2020) Scaling laws for neural language models. CoRR abs/2001.08361

Karpukhin V, Oguz B, Min S, et al (2020) Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 6769–6781

Khandelwal U, Levy O, Jurafsky D, et al (2020) Generalization through memorization: Nearest neighbor language models. In: Proceedings of The 8th International Conference on Learning Representations (ICLR)

Khandelwal U, Fan A, Jurafsky D, et al (2021) Nearest neighbor machine translation. In: Proceedings of the 9th International Conference on Learning Representations (ICLR)

Kim K, Lee J (2024) RE-RAG: improving open-domain QA performance and interpretability with relevance estimator in retrieval-augmented generation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 22149–22161, https://doi.org/10.18653/V1/2024.EMNLP-MAIN.1236

Kim S, Gholami A, Yao Z, et al (2021) I-BERT: integer-only BERT quantization. In: Proceedings of the 38th International Conference on Machine Learning (ICML), pp 5506–5518

King B, Flanigan J (2023) Diverse retrieval-augmented in-context learning for dialogue state tracking. In: Findings of the Association for Computational Linguistics: ACL, pp 5570–5585

Kwiatkowski T, Palomaki J, Redfield O, et al (2019) Natural questions: a benchmark for question answering research. Transactions of the Association for Computational Linguistics 7:452–466

Kwon W, Li Z, Zhuang S, et al (2023) Efficient memory management for large language model serving with pagedattention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), pp 611–626, https://doi.org/10.1145/3600006.3613165

LangChain (2023) LangChain. Software, URL https://www.langchain.com/

LangChain (2026) Langsmith observability. https://docs.langchain.com/langsmith/observability, accessed: 2026-03-04

Lazaridou A, Gribovskaya E, Stokowiec W, et al (2022) Internet-augmented language models through few-shot prompting for open-domain question answering. CoRR abs/2203.05115. https://doi.org/10.48550/ARXIV.2203.05115

Lee K, Han S, Hwang S, et al (2023) When to read documents or QA history: On unified and selective open-domain QA. In: Findings of the Association for Computational Linguistics: ACL, pp 6420–6432

Lee S, Shakir A, Koenig D, et al (2024) Open source strikes bread - new fluffy embeddings model. URL https://www.mixedbread.ai/blog/mxbai-embed-large-v1

Lehman E, Hernandez E, Mahajan D, et al (2023) Do we still need clinical language models? In: Conference on Health, Inference, and Learning (CHIL), pp 578–597

Lewis PSH, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS)

Lewis PSH, Oguz B, Xiong W, et al (2022) Boosted dense retriever. In: Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL), pp 3102–3117

Li H, Su Y, Cai D, et al (2022a) A survey on retrieval-augmented text generation. CoRR abs/2202.01110

Li H, Xu M, Song Y (2023a) Sentence embedding leaks more information than you expect: Generative embedding inversion attack to recover the whole sentence. In: Findings of the Association for Computational Linguistics: ACL, pp 14022–14040

Li M, Wang Y, Zhang P, et al (2023b) Deep learning for approximate nearest neighbour search: A survey and future directions. IEEE Transactions on Knowledge and Data Engineering 35(9):8997–9018

Li X, Li P, Hu P (2023c) Revisiting source context in nearest neighbor machine translation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)

Li X, Lv K, Yan H, et al (2023d) Unified demonstration retriever for in-context learning. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 4644–4668

Li Y, Yang Y, Quan X, et al (2021) Retrieve & memorize: Dialog policy learning with multi-action memory. In: Findings of the Association for Computational Linguistics (ACL/IJCNLP), pp 447–459

Li Y, Wen H, Wang W, et al (2024a) Personal LLM agents: Insights and survey about the capability, efficiency and security. CoRR abs/2401.05459

Li Y, Yue X, Liao Z, et al (2024b) Attributionbench: How hard is automatic attribution evaluation? In: Findings of the Association for Computational Linguistics: ACL, pp 14919–14935

Li Z, Guo R, Kumar S (2022b) Decoupled context processing for context augmented language modeling. In: Advances in Neural Information Processing Systems 35 (NeurIPS)

Liang X, Niu S, Li Z, et al (2025) Saferag: Benchmarking security in retrieval-augmented generation of large language model. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp 4609–4631

Lin BY, Tan K, Miller C, et al (2022) Unsupervised cross-task generalization via retrieval augmentation. In: Advances in Neural Information Processing Systems 35 (NeurIPS)

Liu J (2022) LlamaIndex. https://doi.org/10.5281/zenodo.1234, URL https://github.com/jerryjliu/llama_index

Liu J, Li L, Xiang T, et al (2023a) TCRA-LLM: token compression retrieval augmented large language model for inference cost reduction. In: Findings of the Association for Computational Linguistics: EMNLP, pp 9796–9810, https://doi.org/10.18653/V1/2023.FINDINGS-EMNLP.655

Liu NF, Lin K, Hewitt J, et al (2024) Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics 12:157–173

Liu S, Wei Y (2015) Fast nearest neighbor searching based on improved vp-tree. Pattern Recognition Letters 60-61:8–15

Liu S, Cho H, Freedman M, et al (2023b) RECAP: retrieval-enhanced context-aware prefix encoder for personalized dialogue response generation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 8404–8419

Liu Y, Ott M, Goyal N, et al (2019) Roberta: A robustly optimized BERT pretraining approach. CoRR abs/1907.11692

Liu Y, Iter D, Xu Y, et al (2023c) G-eval: NLG evaluation using gpt-4 with better human alignment. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 2511–2522

LMDB (2014) LMDB. Software, URL https://github.com/LMDB/lmdb

Luo M, Jain S, Gupta A, et al (2023) A study on the efficiency and generalization of light hybrid retrievers. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 1617–1626

Lyu X, Grafberger S, Biegel S, et al (2023) Improving retrieval-augmented large language models via data importance learning. CoRR abs/2307.03027. https://doi.org/10.48550/ARXIV.2307.03027

Malkov YA, Yashunin DA (2020) Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence 42(4):824–836

Manakul P, Liusie A, Gales MJF (2023) Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 9004–9017, https://doi.org/10.18653/V1/2023.EMNLP-MAIN.557

Mao S, Jiang Y, Chen B, et al (2024) Rafe: Ranking feedback improves query rewriting for RAG. In: Findings of the Association for Computational Linguistics: EMNLP, pp 884–901, https://doi.org/10.18653/V1/2024.FINDINGS-EMNLP.49

Martinelli I, Ponti MA (2025) Towards secure retrieval-augmented generation: Preventing LLM data leaks with RBAC. In: Proceedings of the 35th Brazilian Conference on Intelligent Systems (BRACIS), Lecture Notes in Computer Science, vol 16182. Springer, pp 500–515

Martins PH, Marinho Z, Martins AFT (2022) Chunk-based nearest neighbor machine translation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 4228–4245

Mei K, Li Z, Xu S, et al (2024) AIOS: LLM agent operating system. CoRR abs/2403.16971

Meng K, Bau D, Andonian A, et al (2022a) Locating and editing factual associations in GPT. In: Advances in Neural Information Processing Systems 35 (NeurIPS)

Meng Y, Li X, Zheng X, et al (2022b) Fast nearest neighbor machine translation. In: Findings of the Association for Computational Linguistics: ACL, pp 555–565

Mesnard T, Hardin C, Dadashi R, et al (2024) Gemma: Open models based on gemini research and technology. CoRR abs/2403.08295

Microsoft (2024) Retrieval-augmented generation (rag) applications with autogen. https://microsoft.github.io/autogen/0.2/blog/2023/10/18/RetrieveChat/, accessed: 2026-03-04

Microsoft (2025a) Adding retrieval augmented generation (rag) to semantic kernel agents. https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/agent-rag?pivots=programming-language-csharp, accessed: 2026-03-04

Microsoft (2025b) Get started with rag using a prompt flow sample. https://learn.microsoft.com/en-us/azure/machine-learning/how-to-use-retrieval-augmented-generation?view=azureml-api-2, accessed: 2026-03-04

Microsoft (2025c) Overview of the microsoft 365 copilot retrieval api. https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/api/ai-services/retrieval/overview, accessed: 2026-03-04

Microsoft (2026a) Agentic retrieval in azure ai search. https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview, accessed: 2026-03-04

Microsoft (2026b) Retrieval-augmented generation (rag) in azure ai search. https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview, accessed: 2026-03-04

Mihaylov T, Clark P, Khot T, et al (2018) Can a suit of armor conduct electricity? A new dataset for open book question answering. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 2381–2391

Min S, Krishna K, Lyu X, et al (2023a) Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 12076–12100

Min S, Shi W, Lewis M, et al (2023b) Nonparametric masked language modeling. In: Findings of the Association for Computational Linguistics: ACL, pp 2097–2118

Mueller A, Narang K, Mathias L, et al (2023) Meta-training with demonstration retrieval for efficient few-shot learning. In: Findings of the Association for Computational Linguistics: ACL, pp 6049–6064

Muhamed A, Ribeiro LFR, Dreyer M, et al (2025) Refusalbench: Generative evaluation of selective refusal in grounded language models. CoRR abs/2510.10390

Muszynska E (2016) Graph- and surface-level sentence chunking. In: Proceedings of the ACL 2016 Student Research Workshop, pp 93–99

Narayan S, Cohen SB, Lapata M (2018) Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 1797–1807

Ni B, Liu Z, Wang L, et al (2025) Towards trustworthy retrieval augmented generation for large language models: A survey. CoRR abs/2502.06872

NLTK (2001) NLTK. misc, URL https://www.nltk.org/

OpenAI (2022) Text-Emb-Ada. URL https://platform.openai.com/docs/guides/embeddings

OpenAI (2023) GPT-4 technical report. CoRR abs/2303.08774

OpenAI (2026) Evals – openai api reference. https://platform.openai.com/docs/api-reference/evals, accessed: 2026-03-04

Packer C, Fang V, Patil SG, et al (2023) Memgpt: Towards llms as operating systems. CoRR abs/2310.08560

Paranjape B, Lamm M, Tenney I (2022) Retrieval-guided counterfactual generation for QA. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1670–1686

Park J, Cho S, Lee J (2025) Stop-rag: Value-based retrieval control for iterative RAG. CoRR abs/2510.14337. https://doi.org/10.48550/ARXIV.2510.14337

Park S, Lee J (2024) Toward robust ralms: Revealing the impact of imperfect retrieval on retrieval-augmented language models. Transactions of the Association for Computational Linguistics 12:1686–1702

Peng X, Qin C, Chen Z, et al (2025) UNIDOC-BENCH: A unified benchmark for document-centric multimodal RAG. CoRR abs/2510.03663

Petroni F, Rocktäschel T, Riedel S, et al (2019) Language models as knowledge bases? In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language

Processing (EMNLP-IJCNLP), pp 2463–2473

Pipitone N, Alami GH (2024) Legalbench-rag: A benchmark for retrieval-augmented generation in the legal domain. CoRR abs/2408.10343

Radford A, Narasimhan K, Salimans T, et al (2018) Improving language understanding by generative pre-training. OpenAI blog

Radford A, Wu J, Child R, et al (2019) Language models are unsupervised multitask learners. OpenAI blog 1(8):9

Rajaraman A, Ullman JD (2011) Data Mining, Cambridge University Press, p 1–17

Rajpurkar P, Jia R, Liang P (2018) Know what you don't know: Unanswerable questions for squad. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), pp 784–789

Ram O, Bezalel L, Zicher A, et al (2023a) What are you token about? dense retrieval as distributions over the vocabulary. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 2481–2498

Ram O, Levine Y, Dalmedigos I, et al (2023b) In-context retrieval-augmented language models. Transactions of the Association for Computational Linguistics 11:1316–1331. https://doi.org/10.1162/TACL_A_00605

Ram P, Sinha K (2019) Revisiting kd-tree for nearest neighbor search. In: Teredesai A, Kumar V, Li Y, et al (eds) Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), pp 1378–1388

Reid M, Savinov N, Teplyashin D, et al (2024) Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. CoRR abs/2403.05530

Reimers N, Gurevych I (2019) Sentence-bert: Sentence embeddings using siamese bert-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 3980–3990

Ren Y, Cao Y, Guo P, et al (2023) Retrieve-and-sample: Document-level event argument extraction via hybrid retrieval augmentation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 293–306

Robertson SE, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4):333–389

Roy N, Ribeiro LFR, Blloshmi R, et al (2024) Learning when to retrieve, what to rewrite, and how to respond in conversational QA. In: Findings of the Association for Computational Linguistics: EMNLP, pp 10604–10625, https://doi.org/10.18653/

V1/2024.FINDINGS-EMNLP.622

Ru D, Qiu L, Hu X, et al (2024) Ragchecker: A fine-grained framework for diagnosing retrieval-augmented generation. In: Advances in Neural Information Processing Systems 38 (NeurIPS)

Saad-Falcon J, Khattab O, Potts C, et al (2024) ARES: an automated evaluation framework for retrieval-augmented generation systems. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL), pp 338–354

Sachan DS, Reddy S, Hamilton WL, et al (2021) End-to-end training of multi-document reader and retriever for open-domain question answering. In: Advances in Neural Information Processing Systems 34 (NeurIPS), pp 25968–25981

Sanh V, Debut L, Chaumond J, et al (2019) Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. CoRR abs/1910.01108

Schick T, Dwivedi-Yu J, Dessì R, et al (2023) Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Sellam T, Das D, Parikh AP (2020) BLEURT: learning robust metrics for text generation. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). Association for Computational Linguistics, pp 7881–7892

Shao Z, Gong Y, Shen Y, et al (2023) Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In: Findings of the Association for Computational Linguistics: EMNLP, pp 9248–9274

Shen M, Umar M, Maeng K, et al (2024) Towards understanding systems trade-offs in retrieval-augmented generation model inference. CoRR abs/2412.11854

Shi W, Min S, Yasunaga M, et al (2024) REPLUG: retrieval-augmented black-box language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL), pp 8371–8384

Shuster K, Poff S, Chen M, et al (2021) Retrieval augmentation reduces hallucination in conversation. In: Findings of the Association for Computational Linguistics: EMNLP, pp 3784–3803

Siino M, Tinnirello I (2024) GPT hallucination detection through prompt engineering. In: Working Notes of the Conference and Labs of the Evaluation Forum (CLEF), pp 712–721

Siino M, Tinnirello I, Cascia ML (2024) Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on transformers and traditional classifiers. Inf Syst 121:102342

Siino M, Tinnirello I, Cascia ML (2025) From foundations to GPT in text classification: A comprehensive survey on current approaches and future trends. Found Trends Inf Retr 19(5):557–711

Singhal K, Azizi S, Tu T, et al (2023a) Large language models encode clinical knowledge. Nature 620(7972):172–180

Singhal K, Tu T, Gottweis J, et al (2023b) Towards expert-level medical question answering with large language models. CoRR abs/2305.09617

Siriwardhana S, Weerasekera R, Kaluarachchi T, et al (2023) Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering. Transactions of the Association for Computational Linguistics 11:1–17

Spotify (2017) Annoy. Software, URL https://github.com/spotify/annoy

Spring (2026) Retrieval augmented generation-spring ai. https://docs.spring.io/spring-ai/reference/api/retrieval-augmented-generation.html, accessed: 2026-03-04

Su J, Ahmed MHM, Lu Y, et al (2024a) Roformer: Enhanced transformer with rotary position embedding. Neurocomputing 568:127063

Su W, Tang Y, Ai Q, et al (2024b) DRAGIN: dynamic retrieval augmented generation based on the real-time information needs of large language models. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp 12991–13013, https://doi.org/10.18653/V1/2024.ACL-LONG.702

Su W, Tang Y, Ai Q, et al (2025a) Parametric retrieval augmented generation. In: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), pp 1240–1250

Su Y, Collier N (2023) Contrastive search is what you need for neural text generation. Transactions on Machine Learning Research 2023

Su Z, Mo F, Nie J (2025b) Parametric retrieval-augmented generation using latent routing of lora adapters. CoRR abs/2511.17044

Sulem E, Hay J, Roth D (2021) Do we know what we don't know? studying unanswerable questions beyond squad 2.0. In: Findings of the Association for Computational Linguistics: EMNLP, pp 4543–4548

Sun J, Xu C, Tang L, et al (2024) Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In: Proceedings of The Twelfth

International Conference on Learning Representations (ICLR)

Tan Y, He S, Liao H, et al (2025) Dynamic parametric retrieval augmented generation for test-time knowledge enhancement. CoRR abs/2503.23895

Team L (2024) The llama 3 herd of models. CoRR abs/2407.21783

Thakur N, Reimers N, Rücklé A, et al (2021) BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)

Touvron H, Lavril T, Izacard G, et al (2023a) Llama: Open and efficient foundation language models. CoRR abs/2302.13971

Touvron H, Martin L, Stone K, et al (2023b) Llama 2: Open foundation and fine-tuned chat models. CoRR abs/2307.09288

Uddin MN, Saeidi A, Handa D, et al (2025) Unseentimeqa: Time-sensitive question-answering beyond llms' memorization. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp 1873–1913

Vaithilingam P, Zhang T, Glassman EL (2022) Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In: Barbosa SDJ, Lampe C, Appert C, et al (eds) CHI Conference on Human Factors in Computing Systems (CHI), pp 332:1–332:7

Vaswani A, Shazeer N, Parmar N, et al (2017) Attention is all you need. In: Advances in Neural Information Processing Systems 30 (NeurIPS), pp 5998–6008

Vercel (2026) Rag agent guide – vercel ai sdk. https://ai-sdk.dev/cookbook/guides/rag-chatbot, accessed: 2026-03-04

Vu T, Iyyer M, Wang X, et al (2024) Freshllms: Refreshing large language models with search engine augmentation. In: Findings of the Association for Computational Linguistics: ACL, pp 13697–13720, https://doi.org/10.18653/V1/2024.FINDINGS-ACL.813

Wang B, Ping W, Xu P, et al (2023a) Shall we pretrain autoregressive language models with retrieval? A comprehensive study. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 7763–7786

Wang B, Ping W, McAfee L, et al (2024a) Instructretro: Instruction tuning post retrieval-augmented pretraining. In: Proceedings of the Forty-first International Conference on Machine Learning (ICML), pp 51255–51272

Wang J, Yi X, Guo R, et al (2021a) Milvus: A purpose-built vector data management system. In: SIGMOD '21: International Conference on Management of Data. ACM,

pp 2614–2627

Wang L, Ma C, Feng X, et al (2024b) A survey on large language model based autonomous agents. Frontiers in Computer Science 18(6):186345

Wang S, Xu Y, Fang Y, et al (2022) Training data is more valuable than you think: A simple and effective method by retrieving from training data. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), pp 3170–3179

Wang S, Zhu Y, Liu H, et al (2025a) Knowledge editing for large language models: A survey. ACM Computing Surveys 57(3):59:1–59:37

Wang W, Dong L, Cheng H, et al (2023b) Augmenting language models with long-term memory. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Wang W, Zou W, Min J (2025b) Solving LLM repetition problem in production: A comprehensive study of multiple solutions. CoRR abs/2512.04419. https://doi.org/10.48550/ARXIV.2512.04419

Wang X, Jiang Y, Bach N, et al (2021b) Improving named entity recognition by external context retrieving and cooperative learning. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 1800–1812

Wang Y, Li P, Sun M, et al (2023c) Self-knowledge guided retrieval augmentation for large language models. In: Findings of the Association for Computational Linguistics: EMNLP, pp 10303–10315

Wang Z, Araki J, Jiang Z, et al (2023d) Learning to filter context for retrieval-augmented generation. CoRR abs/2311.08377. https://doi.org/10.48550/ARXIV.2311.08377

Wang Z, Nie W, Qiao Z, et al (2023e) Retrieval-based controllable molecule generation. In: The Eleventh International Conference on Learning Representations, (ICLR)

Wornow M, Lozano A, Dash D, et al (2024) Zero-shot clinical trial patient matching with llms. CoRR abs/2402.05125

Wu M, Goodman ND (2018) Multimodal generative models for scalable weakly-supervised learning. In: Advances in Neural Information Processing Systems 31 (NeurIPS), pp 5580–5590

Wu S, Xiong Y, Cui Y, et al (2024) Refusion: Improving natural language understanding with computation-efficient retrieval representation fusion. In: Proceedings of The Twelfth International Conference on Learning Representations (ICLR)

Wu Y, Rabe MN, Hutchins D, et al (2022) Memorizing transformers. In: Proceedings of The Tenth International Conference on Learning Representations (ICLR)

Xi Z, Chen W, Guo X, et al (2025) The rise and potential of large language model based agents: a survey. Science China Information Sciences 68(2)

Xia S, Wang X, Liang J, et al (2025) Ground every sentence: Improving retrieval-augmented llms with interleaved reference-claim generation. In: Findings of the Association for Computational Linguistics (NAACL), pp 969–988

Xiang Z, Wu C, Zhang Q, et al (2025) When to use graphs in RAG: A comprehensive analysis for graph retrieval-augmented generation. CoRR abs/2506.05690

Xiao S, Liu Z, Zhang P, et al (2023) C-pack: Packaged resources to advance general chinese embedding. CoRR abs/2309.07597

Xie J, Chen Z, Zhang R, et al (2024) Large multimodal agents: A survey. CoRR abs/2402.15116

Xin J, Tang R, Yu Y, et al (2021) The art of abstention: Selective prediction and error regularization for natural language processing. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 1040–1051

Xiong G, Jin Q, Lu Z, et al (2024) Benchmarking retrieval-augmented generation for medicine. In: Findings of the Association for Computational Linguistics: ACL, pp 6233–6251

Xiong Y, Yang X, Liu L, et al (2023) EARA: improving biomedical semantic textual similarity with entity-aligned attention and retrieval augmentation. In: Findings of the Association for Computational Linguistics: EMNLP, pp 8760–8771

Xu F, Shi W, Choi E (2024) RECOMP: improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of The Twelfth International Conference on Learning Representations (ICLR)

Xu FF, Alon U, Neubig G (2023) Why do nearest neighbor language models work? In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp 38325–38341

Yan S, Gu J, Zhu Y, et al (2024) Corrective retrieval augmented generation. CoRR abs/2401.15884

Yang A, Yang B, Hui B, et al (2024a) Qwen2 technical report. CoRR abs/2407.10671

Yang A, Yang B, Zhang B, et al (2024b) Qwen2.5 technical report. CoRR abs/2412.15115

Yang Z, Qi P, Zhang S, et al (2018) Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 2369–2380

Yu G, Liu L, Jiang H, et al (2023a) Retrieval-augmented few-shot text classification. In: Findings of the Association for Computational Linguistics: EMNLP, pp 6721–6735

Yu H, Gan A, Zhang K, et al (2024) Evaluation of retrieval-augmented generation: A survey. CoRR abs/2405.07437

Yu X, Jian P, Chen C (2025) Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 14063–14082

Yu Z, Xiong C, Yu S, et al (2023b) Augmentation-adapted retriever improves generalization of language models as generic plug-in. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 2421–2436

Yue X, Wang B, Chen Z, et al (2023) Automatic evaluation of attribution by large language models. In: Findings of the Association for Computational Linguistics: EMNLP, pp 4615–4635

Zeng A, Liu X, Du Z, et al (2023) GLM-130B: an open bilingual pre-trained model. In: The Eleventh International Conference on Learning Representations (ICLR). OpenReview.net

Zeng S, Zhang J, He P, et al (2025) Mitigating the privacy issues in retrieval-augmented generation (RAG) via pure synthetic data. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 24527–24558

Zhang B, Sennrich R (2019) Root mean square layer normalization. In: Advances in Neural Information Processing Systems 32 (NeurIPS), pp 12360–12371

Zhang H, Chen J, Jiang F, et al (2023a) Huatuogpt, towards taming language model to be a doctor. In: Findings of the Association for Computational Linguistics: EMNLP, pp 10859–10885

Zhang J, Muhamed A, Anantharaman A, et al (2023b) Reaugkd: Retrieval-augmented knowledge distillation for pre-trained language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 1128–1136

Zhang N, Yao Y, Tian B, et al (2024) A comprehensive study of knowledge editing for large language models. CoRR abs/2401.01286

Zhang Z, Dai Q, Bo X, et al (2025) A survey on the memory mechanism of large language model-based agents. ACM Transactions on Information Systems

43(6):155:1–155:47

Zhao P, Zhang H, Yu Q, et al (2024) Retrieval-augmented generation for ai-generated content: A survey. CoRR abs/2402.19473

Zheng L, Chiang W, Sheng Y, et al (2023) Judging llm-as-a-judge with mt-bench and chatbot arena. In: Advances in Neural Information Processing Systems 36 (NeurIPS)

Zheng X, Zhang Z, Guo J, et al (2021) Adaptive nearest neighbor machine translation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 368–374, https://doi.org/10.18653/V1/2021.ACL-SHORT.47

Zhong Z, Lei T, Chen D (2022) Training language models with memory augmentation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 5657–5673, https://doi.org/10.18653/V1/2022.EMNLP-MAIN.382

Zhu R, Liu X, Sun Z, et al (2025) Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp 22362–22375

Zhu W, Xu J, Huang S, et al (2023) INK: injecting knn knowledge in nearest neighbor machine translation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 15948–15959

Zou W, Geng R, Wang B, et al (2025) Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. In: Proceedings of the 34th USENIX Security Symposium (USENIX Security), pp 3827–3844